

Project 3.2 - Sales Forecasting Using LightGBM & SHAP

Nguyễn Quốc Thái

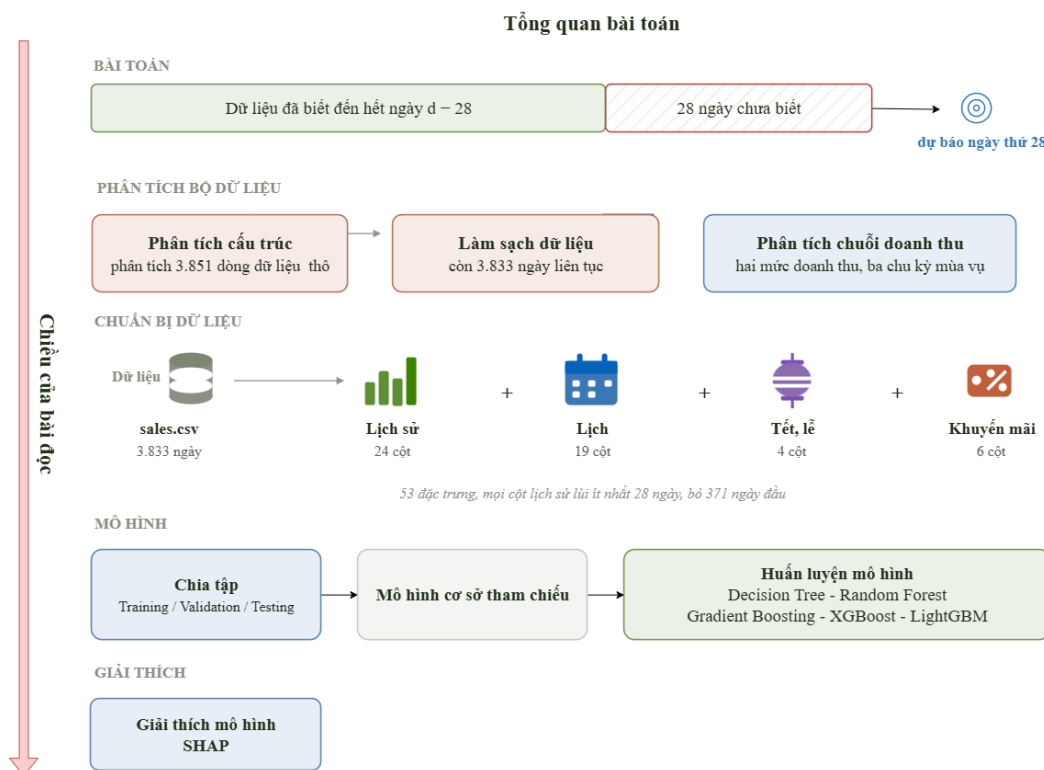
Nguyễn Lê Minh Thông

Đinh Quang Vinh

(Từng câu trong bài viết này đã được duyệt lại bởi nhóm tác giả)

Mở đầu

Dự báo doanh thu là một trong những bài toán được các doanh nghiệp bán lẻ quan tâm từ sớm. Bộ phận kế hoạch cần biết trước doanh thu dự kiến của tháng tới để đặt hàng từ nhà cung cấp, bố trí nhân sự kho và chốt ngân sách quảng cáo.



Hình 1: Tổng quan về bài toán dự đoán doanh thu dựa trên các mô hình học máy: Decision Tree, Random Forest, Gradient Boosting, XGBoost, LightGBM.

Theo Hình 1, trong bài học này chúng ta sẽ xây dựng một mô hình dự báo doanh thu bán lẻ hàng ngày, đi từ bước đọc dữ liệu thô cho tới bước giải thích kết quả. Dữ liệu là doanh thu của một chuỗi thương mại điện tử trong hơn mười năm, từ tháng 7 năm 2012 đến hết năm 2022.

Phát biểu chính xác của bài toán như sau. Với khoảng dự báo $H = 28$ ngày, mục tiêu là xây dựng hàm f để dự đoán doanh thu của ngày d :

$$\hat{y}_d = f(\mathcal{I}_{d-H}, c_d), \quad (1)$$

trong đó $\mathcal{I}_{d-H}$ là toàn bộ dữ liệu doanh thu đã quan sát được đến hết ngày $d - H$, còn c_d là các thông tin của ngày d có thể biết trước, chẳng hạn như thứ trong tuần, ngày trong tháng hoặc lịch khuyến mãi đã được công bố.

Với $H = 28$, khi dự đoán doanh thu cho ngày d , mô hình phải giả sử rằng thời điểm hiện tại đang ở ngày $d - 28$. Nói cách khác, mô hình chỉ được sử dụng dữ liệu doanh thu có sẵn đến hết ngày $d - 28$ và không được nhìn thấy doanh thu của 27 ngày nằm giữa $d - 28$ và d .

Khoảng cách 28 ngày này được gọi là khoảng dự báo (*forecast horizon*). Theo ràng buộc về mốc $d - H$, mọi đặc trưng được tạo từ doanh thu trong quá khứ, chẳng hạn như giá trị trễ (*lag*), trung bình trượt (*moving average*) hoặc tổng doanh thu gần đây, đều phải được tính chỉ từ dữ liệu nằm trước hoặc tại mốc này. Quy tắc này giúp bảo đảm rằng quá trình huấn luyện mô phỏng đúng tình huống dự báo thực tế và tránh việc mô hình vô tình sử dụng thông tin từ tương lai.

Mô hình dùng trong bài là các mô hình cây, gồm cây quyết định, rừng ngẫu nhiên và ba biến thể boosting là Gradient Boosting, XGBoost và LightGBM.

Mục lục

I.	Giới thiệu bài toán	5
I.1.	Các bước giải quyết bài toán	5
I.2.	Bộ dữ liệu	5
II.	Phân tích cấu trúc dữ liệu	8
II.1.	Kiểm tra kiểu dữ liệu	8
II.2.	Kiểm tra định dạng ngày	9
II.3.	Kiểm tra dữ liệu trùng	10
II.4.	Kiểm tra ngày thiếu	11
II.5.	Kiểm tra giá trị thiếu	12
III.	Làm sạch dữ liệu	14
III.1.	Chuyển doanh thu về kiểu số	14
III.2.	Thống nhất định dạng ngày	15
III.3.	Gộp các ngày trùng	16
III.4.	Kiểm tra dữ liệu sau xử lý	17
IV.	Phân tích chuỗi doanh thu	19
IV.1.	Mức doanh thu qua các năm	19
IV.2.	Phân phối doanh thu	21
IV.3.	Các ngày doanh thu cao bất thường	22
IV.4.	Chu kỳ mùa vụ	23
IV.5.	Tết, khuyến mãi và giá vốn	25
IV.6.	Tự tương quan theo độ trễ	28
V.	Xây dựng đặc trưng	31
V.1.	Quy tắc lùi thời gian	31
V.2.	Các nhóm đặc trưng	33
V.3.	Kiểm tra rò rỉ dữ liệu	38
V.4.	Giai đoạn khởi động	39
V.5.	Điểm số khi để rò rỉ lọt vào	40
VI.	Xây dựng mô hình	42

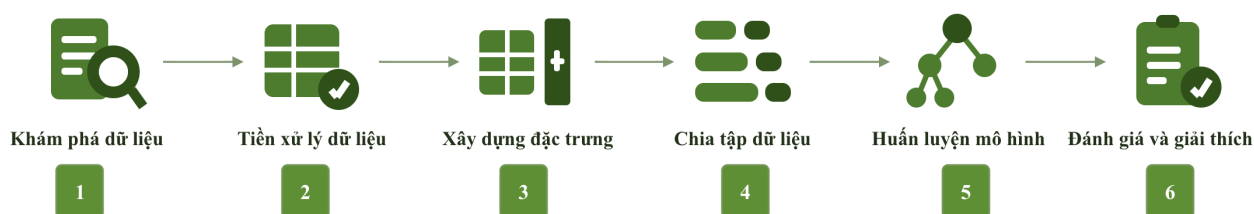
VI.1.	Chia tập theo thời gian	42
VI.2.	Chọn độ đo	44
VI.3.	Mô hình cơ sở	44
VI.4.	Huấn luyện các mô hình cây	47
VI.5.	Đóng góp của từng nhóm đặc trưng	50
VII.	Đánh giá trên tập kiểm tra	52
VII.1.	Kết quả dự báo	52
VII.2.	Thiên lệch của mô hình	53
VIII.	Giải thích mô hình bằng SHAP	56
VIII.1.	Mức quan trọng của từng đặc trưng	56
VIII.2.	Giải thích một dự báo cụ thể	59
IX.	Kết luận	62
Phụ lục		63

I. Giới thiệu bài toán

I.1 Các bước giải quyết bài toán

Ở phần này chúng ta tìm hiểu các bước để giải quyết bài toán dự báo theo quy trình trong Hình 2. Trong suốt quy trình, cần tuân thủ ba điều kiện đó là chia tập trước mọi phép xử lý có sử dụng thống kê, không dùng dữ liệu tương lai kể cả gián tiếp qua một cửa sổ trượt, và chốt mô hình bằng kiểm định chéo chứ không bằng tập kiểm tra. Nếu không thỏa mãn một trong ba điều kiện này, mô hình vẫn có thể cho điểm số tốt khi thử nghiệm nhưng sẽ hoạt động kém khi triển khai thực tế.

Tổng quan các bước thực hiện trong xử lý dữ liệu và huấn luyện mô hình học máy



Hình 2: Quy trình giải quyết bài toán gồm các bước từ xử lý data đến huấn luyện và đánh giá mô hình.

Các mục còn lại của tài liệu đi theo trình tự sau:

- Đọc dữ liệu thô và kiểm tra cấu trúc, ở Mục II.
- Xử lý những lỗi vừa phát hiện, ở Mục III.
- Phân tích chuỗi doanh thu để đưa ra các quyết định thiết kế, ở Mục IV.
- Xây dựng đặc trưng kèm phép thử tự động phát hiện rò rỉ, ở Mục V.
- Chia tập, dựng mô hình cơ sở và huấn luyện năm mô hình cây, ở Mục VI.
- Mở tập kiểm tra một lần duy nhất để đo, ở Mục VII.
- Giải thích mô hình bằng SHAP, ở Mục VIII.

I.2 Bộ dữ liệu

Bộ dữ liệu gồm hai tệp. Tệp chính là `sales.csv` với ba cột. Cột `Date` là ngày, cột `Revenue` là doanh thu, cột `COGS` là giá vốn hàng bán. Biến mục tiêu của tài liệu này là `Revenue`. Một tệp

phụ là `promotions.csv` ghi lịch 50 chương trình khuyến mãi đã công bố, mỗi chương trình có ngày bắt đầu, ngày kết thúc, mức giảm giá và nhóm hàng áp dụng.

Trước khi đọc dữ liệu, chúng ta khai báo một lần các thành phần được sử dụng lại ở những mục sau, gồm khoảng dự báo, cách chia tập theo thời gian, hai hàm chấm điểm và hai hàm đọc, ghi kết quả. Việc tập hợp các thành phần này vào một chỗ giúp những mục sau chỉ chứa nội dung riêng của từng mục.

Khởi tạo

```

1 import json, time
2 import numpy as np, pandas as pd
3
4 H = 28                                # khoảng dự báo: đặt hàng trước bốn tuần
5 TARGET = "revenue"
6 SEED = 42
7
8 # Bốn lần cắt theo cửa sổ mở rộng. Mục VI giải thích vì sao cắt như vậy và
9 # điểm kiểm định chéo chỉ lấy trung bình Fold 2 với Fold 3.
10 FOLDS = [("Fold 1", "2019-05-31", "2019-07-01", "2019-12-31"),
11          ("Fold 2", "2020-05-31", "2020-07-01", "2020-12-31"),
12          ("Fold 3", "2021-05-31", "2021-07-01", "2021-12-31")]
13 TEST_FOLD = ("TEST", "2022-05-31", "2022-07-01", "2022-12-31")
14 CV_FOLDS = ["Fold 2", "Fold 3"]
15
16 def split_by_date(df, train_end, valid_start, valid_end):
17     """Cắt bảng theo mốc thời gian thành train và kiểm định."""
18     return (df[df.date <= train_end],
19             df[(df.date >= valid_start) & (df.date <= valid_end)])
20
21 def metrics(y_true, y_pred):
22     """WAPE là độ đo chính, luôn kèm thiên lệch."""
23     y = np.asarray(y_true, float)
24     p = np.clip(np.asarray(y_pred, float), 0, None)
25     return dict(
26         WAPE=float(np.sum(np.abs(y - p)) / np.sum(y)),
27         bias=float((p.mean() - y.mean()) / y.mean()),
28         R2=float(1 - np.sum((y - p) ** 2)
29                 / np.sum((y - y.mean()) ** 2))
30
31 def save_results(name, obj):
32     """Ghi kết quả ra JSON để mục sau đọc lại."""
33     text = json.dumps(obj, ensure_ascii=False, indent=2,
34                       default=lambda o: o.tolist())
35     open(f"{name}.json", "w", encoding="utf-8").write(text)
36
37 def load_results(name):
38     return json.loads(open(f"{name}.json", encoding="utf-8").read())

```

Năm mô hình cây của Module 03 cũng sẽ được khai báo một lần ở đây, vì cả bước kiểm rò rỉ ở Mục V.5, bước huấn luyện ở Mục VI và bước giải thích ở Mục VIII đều phải chạy trên cùng một cấu hình. Mục VI.4 sẽ so sánh chúng với nhau.

Năm mô hình cây khai báo một lần dùng chung

```

1 def get_models(seed=SEED):
2     """Năm mô hình cây của Module 03, xếp theo độ phức tạp tăng dần."""
3     from sklearn.tree import DecisionTreeRegressor
4     from sklearn.ensemble import (RandomForestRegressor,
5                                   GradientBoostingRegressor)
6     import lightgbm as lgb, xgboost as xgb
7
8     boosting = dict(n_estimators=700, learning_rate=0.04,
9                     subsample=0.85, colsample_bytree=0.85,
10                     reg_lambda=1.0, n_jobs=-1, random_state=seed)
11
12     return {
13         "DecisionTree": DecisionTreeRegressor(
14             max_depth=8, min_samples_leaf=15, random_state=seed),
15         "RandomForest": RandomForestRegressor(
16             n_estimators=400, max_depth=14, min_samples_leaf=3,
17             n_jobs=-1, random_state=seed),
18         "GradientBoosting": GradientBoostingRegressor(
19             n_estimators=400, learning_rate=0.05, max_depth=4,
20             subsample=0.85, random_state=seed),
21         "XGBoost": xgb.XGBRegressor(
22             max_depth=5, tree_method="hist", **boosting),
23         "LightGBM": lgb.LGBMRegressor(
24             num_leaves=31, min_child_samples=20, subsample_freq=5,
25             verbose=-1, **boosting),
26     }

```

Giờ chúng ta tải tệp lên và kiểm tra vài dòng. Ở bước này chúng ta chưa ép kiểu cho cột nào mà đọc tất cả dưới dạng chuỗi, vì nếu để pandas tự đoán kiểu thì nó sẽ tự chọn thay chúng ta và che mất chính những lỗi cần tìm.

Đọc dữ liệu thô dưới dạng chuỗi

```

1 raw = pd.read_csv("sales.csv", dtype=str)
2 print(f"Tệp sales.csv: {len(raw):,} dòng, {raw.shape[1]} cột")
3 print(f"Tên cột: {' ', ' '.join(raw.columns)}")
4 print(raw.head(3).to_string(index=False))

```

Kết quả

```

Tệp sales.csv: 3,851 dòng, 3 cột
Tên cột: Date, Revenue, COGS
  Date      Revenue      COGS
2012-07-04  5123547.94  3982991.19
2012-07-05  2751773.45  2150580.23
2012-07-06  3054029.42  2517632.84

```

II. Phân tích cấu trúc dữ liệu

Ở phần này chúng ta phân tích chi tiết cấu trúc của bộ dữ liệu theo quy trình trong Hình 3. Cả năm bước đều chạy trên bảng vừa đọc dưới dạng chuỗi; mỗi bước kiểm tra một khía cạnh và ghi lại kết quả đạt hoặc không đạt. Tổng cộng có tám phép kiểm. Sau khi hoàn tất, những tiêu chí không đạt sẽ được xử lý ở Mục III.

Quy trình kiểm tra cấu trúc dữ liệu



Hình 3: Các bước kiểm tra dữ liệu.

Để tiện theo dõi, chúng ta dùng chung một hàm `check` ghi lại kết quả của từng bước.

Khung ghi kết quả từng phép kiểm

```

1 def check(name, passed, detail=""):
2     """Ghi lại một phép kiểm. passed = True nghĩa là dữ liệu đạt."""
3     print(f"  {'ĐẠT'} if passed else 'HỎNG'  {name:30s} {detail}")
4
5 raw = pd.read_csv("sales.csv", dtype=str)
6 date_text = raw.Date.astype(str).str.strip()
  
```

II.1 Kiểm tra kiểu dữ liệu

Trước hết chúng ta kiểm tra hai cột `Revenue` và `COGS` có đọc được thành số hay không. Cách làm là ép cả cột sang số bằng `pd.to_numeric` với `errors` đặt thành `"coerce"`, ô nào không ép được sẽ thành rỗng, nên chúng ta chỉ cần đếm số ô rỗng thu về là biết có bao nhiêu ô chứa ký tự lạ.

Đếm số ô không đọc được thành số

```

1 for column in ("Revenue", "COGS"):
2     numeric = pd.to_numeric(raw[column], errors="coerce")
3     n_bad = int(numeric.isna().sum())
4     check(f"{column} đọc được thành số", n_bad == 0,
5           f"{n_bad:,} ô không đọc được")
6
7 bad_value = pd.to_numeric(raw.Revenue, errors="coerce").isna()
8 for value in raw.loc[bad_value, "Revenue"].head(3):
  
```



```

9     print(f"      {value!r}")
10 characters = set("".join(raw.loc[bad_value, "Revenue"]))
11 print(f" Ký tự lạ: {sorted(characters - set('0123456789.'))}")

```

Kết quả

```

HÔNG Revenue đọc được thành số      308 ô không đọc được
HÔNG COGS đọc được thành số          308 ô không đọc được
      '4,612,623.19'
      '1,204,115.66'
      '2,971,078.12'
Ký tự lạ: [',']

```

Kết quả cho thấy 308 ô có dấu phân cách hàng nghìn, chiếm khoảng 8% số dòng, và ký tự lạ duy nhất là dấu phẩy chứ không có ký hiệu tiền tệ nào đi kèm.

Tuy nhiên hậu quả không dừng ở 308 ô đó. Một cột trong pandas chỉ mang một kiểu duy nhất, nên chỉ cần một ô không ép được là cả cột 3,851 dòng giữ kiểu `object` và mọi phép tính số học trên cột đều hỏng theo. Vì vậy chúng ta kiểm tra kiểu dữ liệu trước, rồi mới kiểm các thứ còn lại.

II.2 Kiểm tra định dạng ngày

Tiếp theo chúng ta kiểm tra cột `Date`. Một cột ngày sạch thì dùng chung một định dạng và không kèm phần thời gian, nên chúng ta tách thành hai câu hỏi riêng là cột đang dùng dấu phân cách nào, và có dòng nào chứa dấu hai chấm hay không.

Nhận diện các dạng ngày có trong cột

```

1 has_slash = date_text.str.contains("/", na=False)
2 has_time = date_text.str.contains(":", na=False)
3 n_slash, n_time = int(has_slash.sum()), int(has_time.sum())
4
5 check("Ngày dùng chung một định dạng", n_slash == 0,
6       f"{n_slash:,} gạch chéo, {len(raw) - n_slash:,} gạch ngang")
7 check("Ngày không kèm phần thời gian", n_time == 0,
8       f"{n_time:,} dòng có phần thời gian")
9
10 form = (pd.Series(np.where(has_slash, "dd/mm/yyyy", "yyyy-mm-dd"),
11                  index=raw.index)
12         + np.where(has_time, " hh:mm:ss", ""))
13 print("\n Các dạng gặp trong cột Date:")
14 print(form.value_counts().to_string())

```

Kết quả

```

HÔNG Ngày dùng chung một định dạng 213 gạch chéo, 3,638 gạch ngang
HÔNG Ngày không kèm phần thời gian 214 dòng có phần thời gian

```

Các dạng gặp trong cột Date:

yyyy-mm-dd	3485
yyyy-mm-dd hh:mm:ss	153
dd/mm/yyyy	152
dd/mm/yyyy hh:mm:ss	61

Kết quả cho thấy bốn dạng cùng tồn tại trong một cột, và hai lỗi chồng lên nhau ở 61 dòng vừa dùng dấu gạch chéo vừa kèm phần thời gian.

Trước khi quyết định cách sửa, chúng ta cần xác minh thêm hai điều. Thứ nhất là phần thời gian có thật sự vô nghĩa không, hay có dòng nào mang giờ khác nửa đêm. Thứ hai là nhóm gạch chéo đang viết theo dd/mm/yyyy hay mm/dd/yyyy, vì đọc nhầm sẽ biến ngày 4 tháng 7 thành ngày 7 tháng 4 mà không báo lỗi nào.

Xác minh phần thời gian và thứ tự ngày tháng

```
1 time_part = date_text[has_time].str.split().str[1]
2 all_midnight = (time_part == "00:00:00").all()
3 print(f"Thời gian luôn bằng 00:00:00: {all_midnight}")
4
5 leading = date_text[has_slash].str.slice(0, 2).astype(int)
6 print(f"Số đứng đầu chạy từ {leading.min()} đến {leading.max()}")
```

Kết quả

```
Thời gian luôn bằng 00:00:00: True
Số đứng đầu chạy từ 1 đến 31
```

Phần thời gian luôn bằng nửa đêm nên chúng ta cắt bỏ mà không mất thông tin gì, còn số đứng đầu chạy tới 31 mà không tháng nào mang số 31 nên nhóm gạch chéo chắc chắn viết theo dd/mm/yyyy. Hai kết luận này là căn cứ để chúng ta sửa cột ngày ở Mục III.2.

Lưu ý

Chúng ta không nên dùng `pd.to_datetime(..., dayfirst=True)` cho cả cột khi cột có nhiều định dạng. Cờ này áp cho mọi dòng, kể cả dòng đã viết yyyy-mm-dd, nên 2012-07-04 bị đọc thành ngày 8 tháng 1. Sai lầm này không sinh ra lỗi mà chỉ sinh ra ngày trùng và ngày thiếu ở những chỗ trước đó vốn không có.

II.3 Kiểm tra dữ liệu trùng

Bước thứ ba là kiểm tra xem mỗi ngày có xuất hiện đúng một lần hay không. Bước này yêu cầu cột ngày phải được đọc đúng, nên chúng ta tạm thời xử lý theo cách vừa xác minh: bỏ phần thời gian trước, rồi đọc từng nhóm theo đúng định dạng của nhóm đó.

Đọc tạm cột ngày rồi tìm ngày trùng

```

1 date_only = date_text.str.split().str[0]
2 parsed_date = pd.Series(pd.NaT, index=raw.index, dtype="datetime64[ns]")
3 parsed_date[has_slash] = pd.to_datetime(date_only[has_slash],
4                                     format="%d/%m/%Y")
5 parsed_date[~has_slash] = pd.to_datetime(date_only[~has_slash],
6                                     format="%Y-%m-%d")
7
8 is_duplicated = parsed_date.duplicated(keep=False)
9 n_dup_days = parsed_date[is_duplicated].nunique()
10 check("Không có ngày ghi trùng", is_duplicated.sum() == 0,
11       f"{n_dup_days} ngày, {int(is_duplicated.sum())} dòng")
12
13 dup_rows = raw.loc[is_duplicated, ["Revenue", "COGS"]].assign(
14     d=parsed_date[is_duplicated])
15 identical = dup_rows.duplicated(keep=False).all()
16 print(f" Các dòng trùng giống hệt nhau: {identical}")

```

Kết quả

HÔNG Không có ngày ghi trùng 18 ngày, 36 dòng
 Các dòng trùng giống hệt nhau: True

Kết quả cho thấy 18 ngày bị ghi hai lần, tổng cộng 36 dòng, và các dòng trùng mang giá trị giống hệt nhau nên đây là bản sao thuần túy, gộp lại được an toàn. Nếu chúng mang giá trị khác nhau thì bài toán sẽ khác hẳn, vì khi ấy chúng ta phải quyết định giữ dòng nào, mà quyết định đó cần thông tin nằm ngoài tệp dữ liệu.

II.4 Kiểm tra ngày thiếu

Bước thứ tư là kiểm tra chuỗi ngày có liên tục hay không, vì chuỗi liên tục thì phép độ trễ mới có nghĩa. Nếu chuỗi đứt quãng thì `shift(28)` lùi 28 *dòng* chứ không lùi 28 *ngày*, và mọi đặc trưng dựng trên nó đều sai lệch mà không báo lỗi gì.

Kiểm tra tính liên tục và thứ tự của chuỗi ngày

```

1 unique_date = parsed_date.drop_duplicates().sort_values()
2 gap = unique_date.diff().dropna().dt.days
3 n_gaps = int((gap != 1).sum())
4 is_sorted = parsed_date.is_monotonic_increasing
5 check("Chuỗi ngày liên tục", n_gaps == 0,
6       f"cách lớn nhất {int(gap.max())} ngày, {n_gaps} chỗ đứt")
7 check("Ngày tăng dần theo thứ tự dòng", is_sorted,
8       "đã tăng dần" if is_sorted else "có dòng đảo thứ tự")
9 print(f" Chuỗi trải từ {unique_date.min().date()}")
10     f" đến {unique_date.max().date()}, {len(unique_date):,} ngày")

```

Kết quả

ĐẠT Chuỗi ngày liên tục cách lớn nhất 1 ngày, 0 chỗ đứt
 ĐẠT Ngày tăng dần theo thứ tự dòng đã tăng dần
 Chuỗi trái từ 2012-07-04 đến 2022-12-31, 3,833 ngày

Cả hai phép kiểm đều đạt, chuỗi trái liên tục 3,833 ngày không thiếu ngày nào, và thứ tự dòng trong tệp đã tăng dần theo thời gian. Con số 3,833 này cũng chính là số dòng mà tệp sạch sẽ có, sau khi chúng ta gộp 18 bản sao ở Mục III.

II.5 Kiểm tra giá trị thiếu

Bước cuối cùng là đếm số ô rỗng trên toàn bảng.

Đếm ô rỗng trên toàn bảng

```
1 n_missing = int(raw.isna().sum().sum())
2 check("Không có ô rỗng", n_missing == 0, f"{n_missing} ô rỗng")
```

Kết quả

ĐẠT Không có ô rỗng 0 ô rỗng

Không ô nào rỗng. Cần lưu ý rằng chúng ta đang đếm trên bảng đọc dưới dạng chuỗi nên con số này phản ánh đúng số ô rỗng trong tệp. Nếu đọc với kiểu tự động thì 308 ô có dấu phẩy đã bị biến thành rỗng từ trước, và kết quả sẽ khác hẳn.

Sau khi kiểm tra, dữ liệu có chuỗi thời gian liên tục nên chúng ta không cần chèn ngày thiếu, ngày tăng dần nên không cần sắp xếp lại, và không có ô rỗng nên không cần xử lý dữ liệu thiếu. Đây là ba việc thường tốn nhiều công nhất trong khâu làm sạch, và bộ dữ liệu này không cần làm việc nào.

Bên cạnh đó, Bảng 1 thống kê bốn lỗi còn lại kèm quy mô và hướng xử lý, còn Hình 4 đánh dấu chúng ngay trên bảng dữ liệu thô.

Kết quả kiểm tra dữ liệu thô

Date	Revenue	COGS	
2015-02-27	4695415.58	3896449.69	
2015-03-01	4,612,623.19	3,760,240.57	Doanh thu lưu ở dạng chuỗi
...	...	...	
2015-05-17	6,593,507.34	5,168,476.45	
2015-05-17	6,593,507.34	5,168,476.45	Bản ghi các ngày trùng lặp
...	...	...	
01/10/2015	5,328,206.77	4,579,676.64	Định dạng ngày không thống nhất
01/03/2016 00:00:00	709982.79	623398.17	
2016-05-01 00:00:00	11553720.73	9387796.08	Ngày kèm phần thời gian thừa

Hình 4: Bốn lỗi tìm thấy trong tệp `sales.csv`, đánh dấu trực tiếp trên bảng dữ liệu thô.

Lỗi phát hiện	Quy mô	Hướng xử lý ở Mục
Doanh thu lưu ở dạng chuỗi	308 ô mỗi cột	Bỏ dấu phân cách hàng nghìn rồi ép cả hai cột về số thực
Ngày dùng hai định dạng	213 dòng	Đọc từng nhóm theo đúng định dạng rồi ghép lại
Ngày kèm phần thời gian	214 dòng	Cắt bỏ, vì phần thời gian luôn bằng 00:00:00
Có ngày ghi trùng	18 ngày	Gộp lại, vì các dòng trùng mang giá trị giống hệt nhau

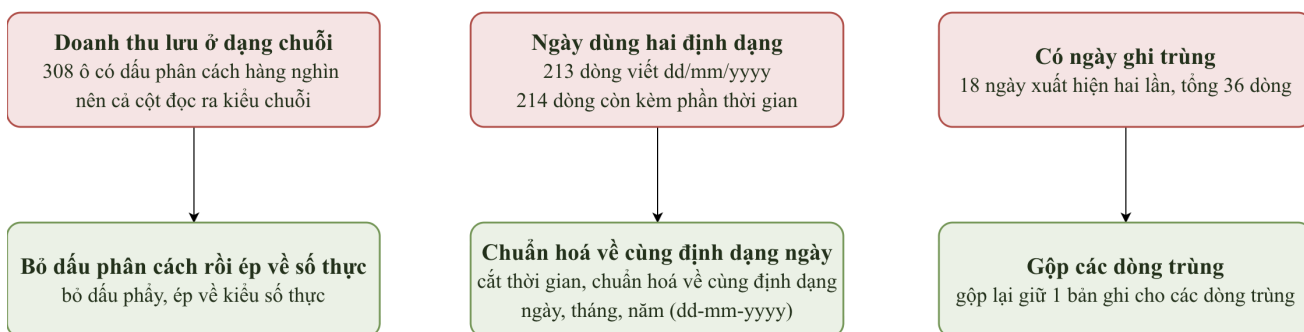
Bảng 1: Thống kê các lỗi cơ bản.

Bốn lỗi này được xử lý bằng ba bước ở Mục III, vì hai lỗi của cột ngày cùng sửa trong một bước. Cả bốn phép sửa đều không làm mất thông tin, vì bỏ dấu phẩy khỏi một con số, đổi định dạng ngày, cắt phần giờ luôn bằng nửa đêm hay gộp hai dòng giống hệt nhau thì đều khôi phục lại được.

III. Làm sạch dữ liệu

Ở phần này chúng ta xử lý các lỗi đã tìm được ở Mục II, theo quy trình vẽ trong Hình 5. Sau khi sửa xong, chúng ta chạy một vòng kiểm tra lại để chắc rằng dữ liệu đã sạch đúng như mong đợi.

Các bước làm sạch dữ liệu



Hình 5: Các bước xử lý, đưa 3,851 dòng thô về 3,833 ngày liên tục.

Trước hết chúng ta đọc lại tệp thô và đổi tên cột sang chữ thường cho thống nhất với phần còn lại của tài liệu.

Đọc lại tệp thô và đổi tên cột

```

1 raw = pd.read_csv("sales.csv", dtype=str)
2 df = raw.rename(columns={"Date": "date", "Revenue": "revenue",
3                           "COGS": "cogs"})
4 n_input = len(df)
5 print(f"Đọc vào {n_input:,} dòng từ sales.csv")
  
```

Kết quả

Đọc vào 3,851 dòng từ sales.csv

III.1 Chuyển doanh thu về kiểu số

Bước đầu tiên là bỏ dấu phân cách hàng nghìn rồi ép hai cột về số thực. Ở Mục II.1 chúng ta đã xác định ký tự lạ duy nhất là dấu phẩy, nên chỉ cần bỏ đúng dấu phẩy mà không đụng đến dấu chấm thập phân.

Bỏ dấu phân cách rồi ép về số thực

```

1 for column in ("revenue", "cogs"):
2     text = df[column].astype(str).str.strip()
3     n_string = int(text.str.contains(",", regex=False).sum())
4     df[column] = text.str.replace(",", "", regex=False).astype(float)
5     print(f" {column:8s}: bỏ dấu phân cách ở {n_string:,} ô,"
6           f" kiểu cột thành {df[column].dtype}")
7
8 print(f" Khoảng giá trị: {df.revenue.min():,.0f}"
9       f" đến {df.revenue.max():,.0f}")

```

Kết quả

```

revenue : bỏ dấu phân cách ở 308 ô, kiểu cột thành float64
cogs     : bỏ dấu phân cách ở 308 ô, kiểu cột thành float64
Khoảng giá trị: 279,814 đến 20,905,271

```

Cả hai cột này mang kiểu float64. Chúng ta in luôn khoảng giá trị để đối chiếu chéo: con số thu về nằm trong khoảng vài trăm nghìn đến hai mươi triệu, cùng cỡ độ lớn với ba dòng mẫu ở Mục I.2.

Lưu ý

Một lỗi thường gặp ở bước này là gọi `str.replace(".", "")` cho cả cột với ý định bỏ dấu phân cách kiểu châu Âu. Trong bộ dữ liệu này dấu chấm lại là dấu thập phân, nên lệnh đó sẽ nhân mọi giá trị lên 100 lần mà không báo lỗi gì. Vì vậy chúng ta luôn xác định ký tự cần bỏ trước, rồi mới bỏ đúng ký tự đó.

III.2 Thống nhất định dạng ngày

Bước thứ hai xử lý cả hai lỗi của cột ngày. Chúng ta cắt phần thời gian trước, rồi mới đọc từng nhóm theo đúng định dạng của nó. Thứ tự này quan trọng, nếu đọc trước khi cắt thì chuỗi 01/03/2016 00:00:00 không khớp với định dạng `%d/%m/%Y` và pandas sẽ báo lỗi.

Vậy nên, trước khi cắt thì chúng ta khẳng định lại bằng một câu `assert` rằng mọi phần thời gian đều bằng nửa đêm. Nếu sau này ai đó thay tệp dữ liệu bằng một bản có giờ thật, câu lệnh sẽ dừng ngay thay vì tự động loại bỏ mất thông tin.

Cắt phần thời gian kèm chốt chặn an toàn

```

1 date_text = df.date.astype(str).str.strip()
2 has_time = date_text.str.contains(":", na=False)
3
4 time_part = date_text[has_time].str.split().str[1]
5 distinct_time = sorted(time_part.unique())
6 print(f" {int(has_time.sum()):,} dòng có phần thời gian,"
7       f" giá trị gặp được: {distinct_time}")

```

```

8 assert distinct_time == ["00:00:00"], "có giờ khác 0, không cắt"
9
10 date_text = date_text.str.split().str[0]

```

Kết quả

214 dòng có phần thời gian, giá trị gộp được: ['00:00:00']

Tiếp theo chúng ta đọc cột ngày. Cách làm là dựng một cột rỗng kiểu ngày tháng, rồi điền vào từng nhóm bằng đúng định dạng của nhóm đó. Nhờ vậy mỗi dòng chỉ được đọc bằng một định dạng duy nhất, và không dòng nào phải để pandas đoán.

Điền cột ngày rỗng theo từng nhóm định dạng

```

1 has_slash = date_text.str.contains("/", na=False)
2 n_slash = int(has_slash.sum())
3
4 parsed = pd.Series(pd.NaT, index=date_text.index,
5                    dtype="datetime64[ns]")
6 parsed[has_slash] = pd.to_datetime(date_text[has_slash],
7                                   format="%d/%m/%Y")
8 parsed[~has_slash] = pd.to_datetime(date_text[~has_slash],
9                                   format="%Y-%m-%d")
10 df["date"] = parsed
11 df = df.sort_values("date").reset_index(drop=True)
12
13 print(f" Đọc {n_slash:,} dòng theo dd/mm/yyyy"
14       f" và {n_input - n_slash:,} dòng theo yyyy-mm-dd")
15 print(f" Chuỗi trái từ {df.date.min().date()}")
16       f" đến {df.date.max().date()}")

```

Kết quả

Đọc 213 dòng theo dd/mm/yyyy và 3,638 dòng theo yyyy-mm-dd
Chuỗi trái từ 2012-07-04 đến 2022-12-31

Sau bước này cột `date` mang kiểu ngày tháng thật, và khi ghi ra tệp nó sẽ chỉ còn một định dạng duy nhất là `yyyy-mm-dd`.

III.3 Gộp các ngày trùng

Bước cuối là gộp 18 ngày bị ghi hai lần. Ở Mục II.3 chúng ta đã xác nhận các dòng trùng mang giá trị giống hệt nhau, nên giữ dòng nào cũng cho kết quả như nhau và chúng ta chỉ cần giữ lại một dòng cho mỗi ngày.

Gộp các dòng trùng ngày

```

1 is_duplicated = df.date.duplicated(keep=False)
2 n_rows = int(is_duplicated.sum())
3 n_days = int(df.date[is_duplicated].nunique())
4 identical = bool(df[is_duplicated].duplicated(keep=False).all())
5
6 print(f"  Tìm thấy {n_days} ngày ghi trùng, tổng {n_rows} dòng")
7 print(f"  Các dòng trùng giống hệt nhau: {identical}")
8
9 df = df.drop_duplicates(subset="date", keep="last")
10 df = df.reset_index(drop=True)
11 print(f"  Còn lại {len(df):,} dòng sau khi gộp")

```

Kết quả

```

Tìm thấy 18 ngày ghi trùng, tổng 36 dòng
Các dòng trùng giống hệt nhau: True
Còn lại 3,833 dòng sau khi gộp

```

Con số 3,833 dòng trùng khớp với số ngày khác nhau mà chúng ta đã đếm được ở Mục II.4, nên phép gộp đã chạy đúng như dự kiến.

III.4 Kiểm tra dữ liệu sau xử lý

Ba bước trên đều sửa dữ liệu, nên chúng ta chạy lại các phép kiểm để xác nhận cả những lỗi đã sửa lẫn những tính chất vốn đã đạt ở Mục II vẫn còn nguyên.

Bảy phép kiểm sau xử lý

```

1 gap = df.date.diff().dropna().dt.days
2 checks = {
3     "số dòng khớp số ngày khác nhau": len(df) == df.date.nunique(),
4     "kiểu doanh thu là số thực": df.revenue.dtype.kind == "f",
5     "kiểu giá vốn là số thực": df.cogs.dtype.kind == "f",
6     "không sinh ô rỗng": int(df.isna().sum().sum()) == 0,
7     "ngày tăng dần": bool(df.date.is_monotonic_increasing),
8     "chuỗi ngày liên tục": bool((gap == 1).all()),
9     "doanh thu đều dương": bool((df.revenue > 0).all()),
10 }
11 for name, passed in checks.items():
12     check(name, passed)
13 assert all(checks.values()), "kiểm tra sau xử lý thất bại"
14
15 df.to_csv("sales_clean.csv", index=False)
16 print(f"\n  {n_input:,} dòng vào, {len(df):,} dòng ra")

```

Kết quả

ĐẠT số dòng khớp số ngày khác nhau
ĐẠT kiểu doanh thu là số thực
ĐẠT kiểu giá vốn là số thực
ĐẠT không sinh ô rỗng
ĐẠT ngày tăng dần
ĐẠT chuỗi ngày liên tục
ĐẠT doanh thu đều dương

3,851 dòng vào, 3,833 dòng ra

Cả bảy tiêu chí đều đạt, và chênh lệch 18 dòng giữa đầu vào với đầu ra đúng bằng số bản sao chúng ta vừa gộp. Câu **assert** ở cuối đóng vai trò như một chốt chặn: nếu sau này có người sửa mã ở một trong ba bước trên và làm hỏng một tính chất nào đó, chương trình sẽ dừng ngay tại đây thay vì để lỗi tiếp tục ảnh hưởng đến các mục sau.

Tệp `sales_clean.csv` thu về là đầu vào cho toàn bộ phần còn lại của tài liệu. Từ Mục IV trở đi, mọi con số đều tính trên 3,833 ngày này.

IV. Phân tích chuỗi doanh thu

Ở phần này chúng ta phân tích chuỗi doanh thu để hiểu nó biến động theo quy luật nào. Khác với Mục II vốn chỉ hỏi tệp dữ liệu có đọc được hay không, phần này hỏi về chính hiện tượng bán lẻ nằm sau con số.

Trước hết chúng ta đọc tệp đã làm sạch và ghi lại vài thuộc tính chung của bộ dữ liệu.

Đọc tệp đã làm sạch

```
1 sales = pd.read_csv("sales_clean.csv", parse_dates=["date"])
2 revenue = sales.revenue
3 print(f"Đọc {len(sales):,} dòng từ sales_clean.csv,"
4       f" từ {sales.date.min().date()} đến {sales.date.max().date()}")
```

Kết quả

Đọc 3,833 dòng từ sales_clean.csv, từ 2012-07-04 đến 2022-12-31

Thuộc tính	Giá trị	Ý nghĩa với bài toán
Số ngày	3,833	Đủ dài để quan sát mười chu kỳ năm
Khoảng thời gian	04/07/2012 – 31/12/2022	Liên tục, không thiếu ngày
Số ngày doanh thu bằng 0	0	Không phải bài toán nhu cầu gián đoạn
Doanh thu trung vị	3,647,304	Đơn vị quy ước của bộ dữ liệu
Khoảng giá trị	279,814 – 20,905,271	Chênh 75 lần nên MAPE không phù hợp
Hệ số biến thiên	0.612	Biến động mạnh, nên độ đo phải bền với ngày cực trị

Bảng 2: Sáu thuộc tính của bộ dữ liệu sạch. Hai dòng cuối, khoảng giá trị chênh 75 lần và hệ số biến thiên 0.612, là lý do chọn WAPE thay cho MAPE hay RMSE.

IV.1 Mức doanh thu qua các năm

Trước hết chúng ta xem chuỗi có giữ một mức ổn định qua các năm hay không, vì nếu mức doanh thu đứt gãy giữa chừng thì cách chia tập và cách đọc kết quả kiểm định đều phải đổi theo.

Doanh thu trung bình ngày theo từng năm

```

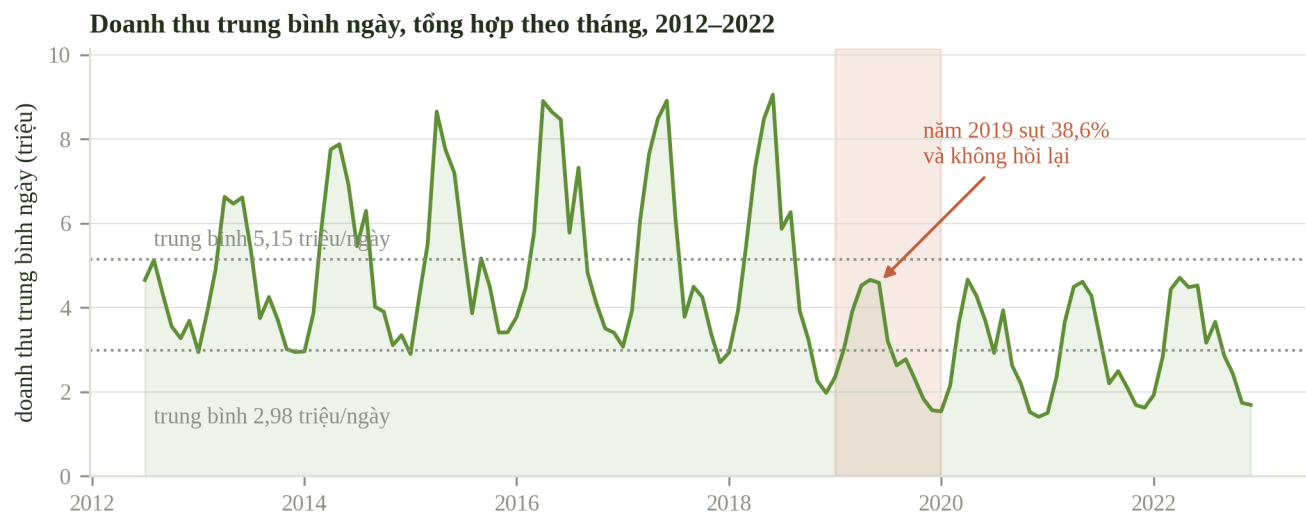
1 by_year = sales.groupby(sales.date.dt.year).revenue.agg(
2     daily_mean="mean", n_days="count")
3 by_year["change_pct"] = by_year.daily_mean.pct_change() * 100
4 print(by_year.round(1).rename_axis("năm").to_string())

```

Kết quả

	daily_mean	n_days	change_pct
năm			
2012	4096672.6	181	NaN
2013	4540190.2	365	10.8
2014	5128344.9	365	13.0
2015	5177900.9	365	1.0
2016	5750384.4	366	11.1
2017	5236066.6	365	-8.9
2018	5068828.6	365	-3.2
2019	3114524.5	365	-38.6
2020	2881180.8	366	-7.5
2021	2857643.3	365	-0.8
2022	3204791.3	365	12.1

Kết quả cho thấy năm 2019 sụt 38.6% so với năm trước, và mức thấp đó không hồi lại trong ba năm tiếp theo. Nếu lấy trung bình giai đoạn 2013 đến 2018 so với giai đoạn 2020 đến 2022 thì hai mức chênh nhau 1.73 lần. Nói cách khác, chuỗi này có hai miền chứ không phải một, và năm 2019 là chỗ chuyển giữa hai miền đó. Hình 6 vẽ lại điều này theo tháng.



Hình 6: Doanh thu trung bình ngày theo tháng. Vùng tô đậm là năm 2019, chỗ chuỗi rớt từ miền 5.15 xuống 2.98 triệu một ngày và không hồi lại.

Quan sát này dẫn tới một quyết định ở Mục VI đó là lần cắt kiểm định rơi vào năm 2019 sẽ được tách riêng, không gộp chung khi tính điểm trung bình. Lý do là mô hình huấn luyện trên miền cao rồi đem đo trên miền thấp thì sai số phản ánh cú đứt gãy chứ không phản ánh chất lượng mô hình.

Một chi tiết nhỏ cần lưu ý là năm 2012 chỉ có 181 ngày, toàn bộ nằm ở nửa cuối năm. Vì vậy con số trung bình của năm 2012 không so sánh trực tiếp được với các năm sau, và cột `change_pct` của năm 2013 cũng cần đọc thật trọng.

IV.2 Phân phối doanh thu

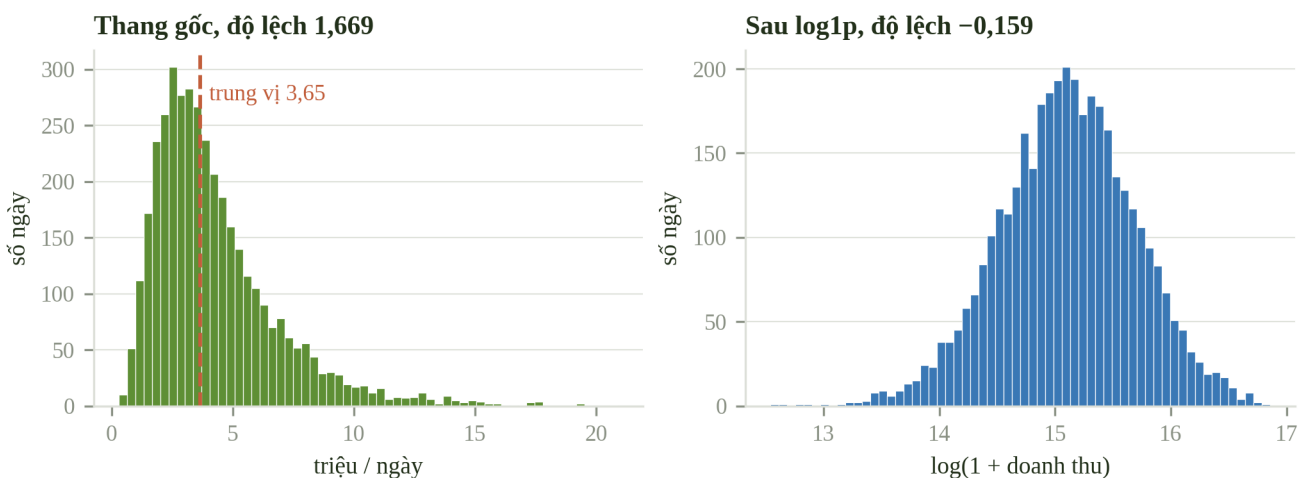
Tiếp theo chúng ta xem doanh thu phân bố như thế nào, vì hình dạng phân phối quyết định hai thứ: có nên đổi thang trước khi huấn luyện hay không, và nên chọn độ đo nào.

Độ lệch trước và sau khi đổi thang log

```
1 from scipy import stats
2
3 print(f" Ngày doanh thu âm: {int((revenue < 0).sum())}")
4     f"   ngày bằng 0: {int((revenue == 0).sum())}")
5 skew_raw = float(stats.skew(revenue))
6 skew_log = float(stats.skew(np.log1p(revenue)))
7 print(f" Độ lệch thang gốc : {skew_raw:+.3f}")
8 print(f" Độ lệch sau log1p : {skew_log:+.3f}")
9 max_over_median = revenue.max() / revenue.median()
10 print(f" Lớn nhất chia trung vị: {max_over_median:.2f} lần")
```

Kết quả

```
Ngày doanh thu âm: 0   ngày bằng 0: 0
Độ lệch thang gốc : +1.669
Độ lệch sau log1p : -0.159
Lớn nhất chia trung vị: 5.73 lần
```



Hình 7: Phân phối lệch phải với độ lệch 1.669, nhưng sau khi đổi sang thang log thì gần như đối xứng, độ lệch còn -0.159 .

Hình 7 đặt hai thang cạnh nhau. Trên thang gốc, phân phối lệch phải với độ lệch 1.669 và ngày cao nhất gấp 5.73 lần trung vị. Sau khi đổi sang thang log thì độ lệch còn -0.159 , tức gần như đối xứng. Vì vậy chúng ta sẽ huấn luyện mô hình trên $\log(1 + y)$ rồi đổi ngược lại khi dự báo, để vài ngày cực trị không kéo lệch toàn bộ quá trình học.

Phân phối lệch cũng loại RMSE và MAPE khỏi danh sách độ đo, nên chúng ta chọn WAPE, với lý do đầy đủ ở Mục VI. Cách chọn độ đo cho bài toán dự báo được bàn kỹ trong Hyndman (2021).

IV.3 Các ngày doanh thu cao bất thường

Ở phần này chúng ta xem những ngày doanh thu cao hẳn so với phần còn lại, để biết chúng là lỗi ghi chép hay là hiện tượng thật. Nếu là lỗi thì chúng ta cắt đi, còn nếu là hiện tượng thật mà cắt thì mô hình sẽ học sai.

Ngưỡng Tukey và tỉ lệ ngày cao rơi vào cụm giao tháng

```

1 q1, q3 = revenue.quantile([0.25, 0.75])
2 upper = q3 + 1.5 * (q3 - q1)
3 is_high = revenue > upper
4
5 end_of_month = sales.date.dt.day.isin([29, 30, 31, 1, 2])
6 share_eom = float(end_of_month[is_high].mean())
7
8 print(f" Ngưỡng Tukey phía trên: {upper:.0f}")
9 print(f" Vượt ngưỡng: {int(is_high.sum())} ngày"
10       f" ({100 * is_high.mean():.1f}%)")
11 print(f" Tỉ lệ rơi vào cụm ngày 29, 30, 31, 1 và 2:"
12       f" {100 * share_eom:.1f}%)")

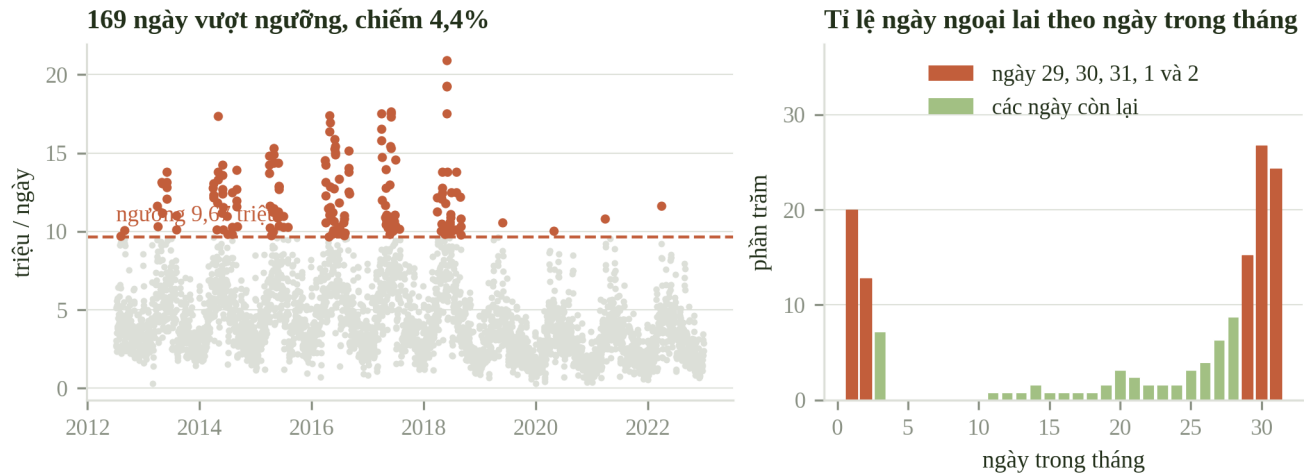
```

Kết quả

```

Ngưỡng Tukey phía trên: 9,670,560
Vượt ngưỡng: 169 ngày (4.4%)
Tỉ lệ rơi vào cụm ngày 29, 30, 31, 1 và 2: 63.9%

```



Hình 8: 169 ngày vượt ngưỡng Tukey, chiếm 4.4% chuỗi, và 63.9% trong số đó rơi vào cụm ngày 29, 30, 31, mùng 1 và mùng 2.

Hình 8 cho thấy 169 ngày vượt ngưỡng, chiếm 4.4% chuỗi, và 63.9% trong số đó rơi vào cụm ngày 29, 30, 31, mùng 1 và mùng 2. Các ngày cao không rải rác ngẫu nhiên mà tụ quanh chỗ giao tháng, nên đây là hiện tượng có quy luật chứ không phải lỗi ghi chép.

Vì vậy chúng ta giữ nguyên toàn bộ 169 ngày và bổ sung cho mô hình một đặc trưng đếm ngược đến cuối tháng ở Mục V để mô hình học quy luật đó.

Lưu ý

Loại bỏ giá trị ngoại lai là một thao tác quen thuộc trong xử lý dữ liệu, nhưng có thể gây hại đối với chuỗi thời gian. Những ngày có doanh thu cao thường đặc biệt quan trọng với người sử dụng kết quả dự báo. Nếu các ngày này tuân theo quy luật, chúng ta cần mô tả quy luật đó bằng một đặc trưng thay vì loại bỏ các quan sát chứa thông tin cần học.

IV.4 Chu kỳ mùa vụ

Chuỗi trải 10.5 chu kỳ năm nên đủ dài để quan sát mùa vụ. Chúng ta đo ba chu kỳ thường gặp trong bán lẻ là ngày trong tháng, tháng trong năm và thứ trong tuần. Để so sánh được giữa các năm có miền khác nhau, mỗi ngày được chia cho trung bình của chính năm đó trước khi lấy trung bình theo nhóm.

Biên độ của ba chu kỳ mùa vụ

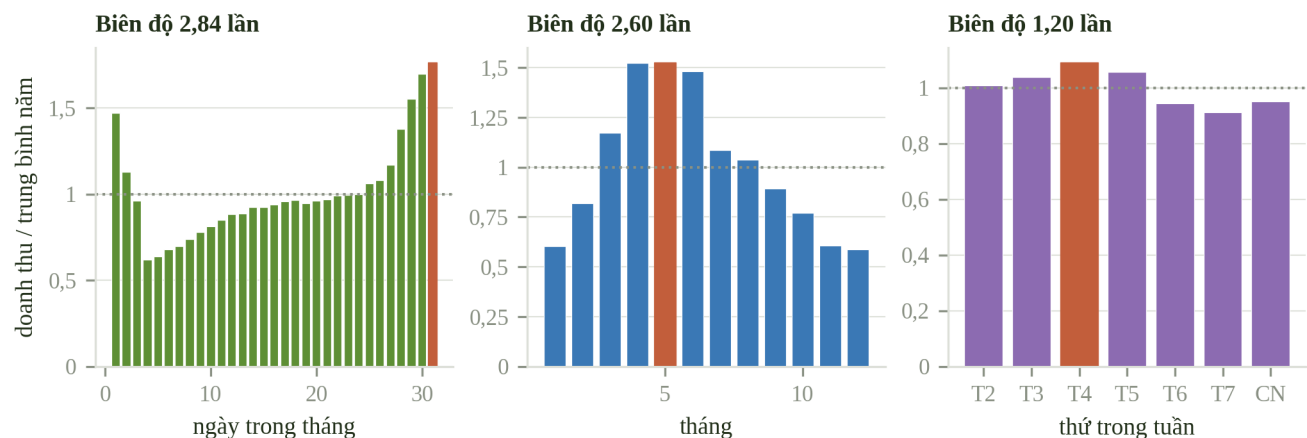
```

1 sales["year_ratio"] = revenue / sales.groupby(
2     sales.date.dt.year).revenue.transform("mean")
3
4 cycles = {}
5 for name, key in [("ngày trong tháng", sales.date.dt.day),
6                  ("tháng", sales.date.dt.month),
7                  ("thứ trong tuần", sales.date.dt.dayofweek)]:
8     g = sales.groupby(key).year_ratio.mean()
9     cycles[name] = {"peak_at": int(g.idxmax()), "peak": float(g.max()),
10                   "trough_at": int(g.idxmin()),
11                   "trough": float(g.min()),
12                   "amplitude": float(g.max() / g.min())}
13 print(pd.DataFrame(cycles).T.round(3))
14 .astype({"peak_at": int, "trough_at": int}).to_string()

```

Kết quả

	peak_at	peak	trough_at	trough	amplitude
ngày trong tháng	31	1.763	4	0.620	2.844
tháng	5	1.525	12	0.586	2.603
thứ trong tuần	2	1.090	5	0.911	1.197



Hình 9: Ba chu kỳ mùa vụ. Cột tô khác màu là đỉnh của mỗi chu kỳ.

Theo Hình 9, chu kỳ ngày trong tháng mạnh nhất với biên độ 2.84 lần giữa đỉnh và đáy, chu kỳ tháng đứng thứ hai với 2.60 lần, còn chu kỳ tuần gần như không đáng kể với 1.20 lần. Kết quả này thống nhất với những gì chúng ta thấy ở Mục IV.3, nơi các ngày cao tụ lại quanh chỗ giao tháng.

Tuy nhiên, trước khi tin vào con số 2.84, chúng ta cần đối chiếu chéo thêm một lần. Ngày 31 chỉ xuất hiện trong 7/12 tháng, và bảy tháng đó không phải là một mẫu ngẫu nhiên đại diện cho cả năm. Vì vậy, đỉnh của chu kỳ này có thể bị nhầm với ảnh hưởng của tháng. Để loại trừ khả năng đó, chúng ta tính lại tỉ lệ của ngày 31 theo hai cách chuẩn hóa khác nhau.

Đối chiếu chéo đỉnh ngày 31 bằng hai cách chuẩn hoá

```

1 sales["month_ratio"] = revenue / sales.groupby(
2     sales.date.dt.to_period("M")).revenue.transform("mean")
3
4 is_31 = sales.date.dt.day == 31
5 by_year_ratio = sales.loc[is_31, "year_ratio"].mean()
6 by_month_ratio = sales.loc[is_31, "month_ratio"].mean()
7 print(f" chuẩn hoá theo năm : {by_year_ratio:.3f}")
8 print(f" chuẩn hoá theo tháng: {by_month_ratio:.3f}")

```

Kết quả

```

chuẩn hoá theo năm : 1.763
chuẩn hoá theo tháng: 1.784

```

Hai con số gần như bằng nhau, nên đỉnh của ngày 31 là hiệu ứng thực sự của ngày trong tháng chứ không phải ảnh hưởng của tháng. Từ đó, chúng ta quyết định bổ sung vào nhóm đặc trưng lịch một cột đếm ngược đến cuối tháng, thay vì chỉ dùng số thứ tự ngày.

IV.5 Tết, khuyến mãi và giá vốn

Ba nguồn tín hiệu còn lại đều nằm ngoài chuỗi doanh thu, gồm lịch âm, lịch khuyến mãi và cột giá vốn. Chúng ta xem xét lần lượt từng nguồn để quyết định có đưa vào mô hình hay không.

IV.5.1 Hiệu ứng Tết

Tết Nguyên đán trượt theo lịch dương nên không nằm cố định ở một ngày nào trong năm. Chúng ta đo bằng cách tính khoảng cách từ mỗi ngày đến mừng 1 Tết gần nhất, rồi lấy trung bình theo bốn cửa sổ quanh Tết. Cũng như ở Mục IV.4, chúng ta chuẩn hoá theo hai cách để phân biệt hiệu ứng Tết thật với hiệu ứng tháng.

Doanh thu ở bốn cửa sổ quanh Tết

```

1 TET_DATES = ["2012-01-23", "2013-02-10", "2014-01-31", "2015-02-19",
2             "2016-02-08", "2017-01-28", "2018-02-16", "2019-02-05",
3             "2020-01-25", "2021-02-12", "2022-02-01", "2023-01-22"]
4
5 tet = pd.to_datetime(TET_DATES)
6 offset = np.stack([(sales.date - t).dt.days.values for t in tet])
7 sales["days_to_tet"] = offset[np.abs(offset).argmin(axis=0),
8                               np.arange(len(sales))]
9
10 tet_table = {}
11 for name, lo, hi in [("30 đến 15 ngày trước", -30, -15),
12                     ("14 đến 1 ngày trước", -14, -1),
13                     ("mùng 1 đến mùng 7", 0, 7),

```

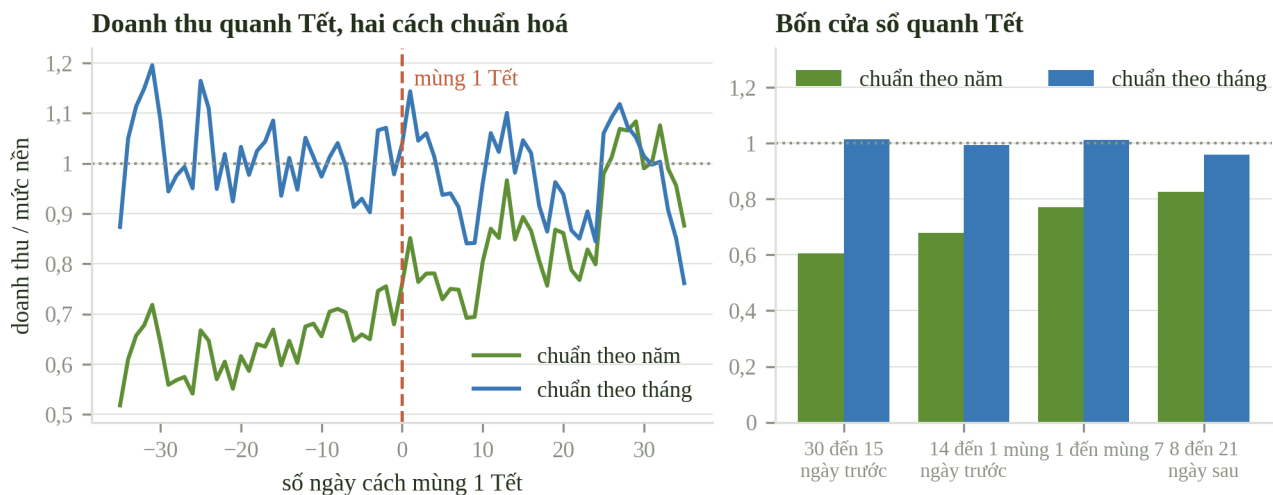
```

14         ("8 đến 21 ngày sau", 8, 21]):
15     w = sales.days_to_tet.between(lo, hi)
16     tet_table[name] = {
17         "by_year": float(sales.loc[w, "year_ratio"].mean()),
18         "by_month": float(sales.loc[w, "month_ratio"].mean())}
19     print(pd.DataFrame(tet_table).T.round(3).to_string())

```

Kết quả

	by_year	by_month
30 đến 15 ngày trước	0.604	1.014
14 đến 1 ngày trước	0.680	0.993
mùng 1 đến mùng 7	0.771	1.011
8 đến 21 ngày sau	0.826	0.959



Hình 10: Doanh thu quanh Tết theo hai cách chuẩn hoá. Chuẩn theo năm cho thấy mức thấp rõ rệt, nhưng chuẩn theo tháng thì cả bốn cửa sổ đều xấp xỉ 1.

Hai cột số trong bảng, vẽ lại ở Hình 10, kể hai câu chuyện khác hẳn nhau. Chuẩn hoá theo năm cho thấy giai đoạn quanh Tết thấp rõ rệt, chỉ bằng 0.604 đến 0.826 mức trung bình năm. Nhưng chuẩn hoá theo tháng thì cả bốn cửa sổ đều xấp xỉ 1, nghĩa là so với chính tháng chứa nó thì giai đoạn quanh Tết không có gì đặc biệt.

Kết luận rút ra là phần lớn cái mà chúng ta tưởng là hiệu ứng Tết thật ra là hiệu ứng tháng, vì Tết luôn rơi vào tháng 1 hoặc tháng 2, mà hai tháng đó vốn đã thấp theo chu kỳ mùa vụ, lần lượt bằng 0.60 và 0.82 mức trung bình năm như Hình 9 cho thấy. Chúng ta vẫn giữ nhóm đặc trưng Tết trong mô hình, nhưng với kỳ vọng đóng góp thấp, và Mục VIII sẽ cho biết kỳ vọng đó có đúng hay không.

IV.5.2 Chương trình khuyến mãi

Tệp `promotions.csv` ghi 50 chương trình khuyến mãi. Điểm quan trọng của nguồn dữ liệu này là lịch được công bố trước, nên tại thời điểm chạy dự báo chúng ta đã biết ngày d có khuyến mãi hay không. Đây là loại thông tin dùng được mà không vi phạm mốc $d - H$.

So sánh ngày có và không có khuyến mãi

```

1 promotions = pd.read_csv("promotions.csv",
2                           parse_dates=["start_date", "end_date"])
3 sales["has_promo"] = 0
4 for _, promo in promotions.iterrows():
5     w = ((sales.date >= promo.start_date)
6          & (sales.date <= promo.end_date))
7     sales.loc[w, "has_promo"] = 1
8
9 with_promo = sales.loc[sales.has_promo == 1, "month_ratio"].mean()
10 without_promo = sales.loc[sales.has_promo == 0, "month_ratio"].mean()
11 print(f" {len(promotions)} chương trình, phủ từ"
12       f" {promotions.start_date.min().date()}"
13       f" đến {promotions.end_date.max().date()}")
14 print(f" ngày có khuyến mãi : {with_promo:.3f}")
15 print(f" ngày không khuyến mãi: {without_promo:.3f}")
16 print(f" chênh lệch {100 * (with_promo / without_promo - 1):+.1f}%")

```

Kết quả

```

50 chương trình, phủ từ 2013-01-31 đến 2022-12-31
ngày có khuyến mãi : 1.010
ngày không khuyến mãi: 0.992
chênh lệch +1.9%

```

Chênh lệch chỉ 1.9%, vẽ ở khung trái của Hình 11, tức khuyến mãi trong bộ dữ liệu này nâng doanh thu rất ít. Chúng ta vẫn đưa nhóm khuyến mãi vào mô hình vì lịch phủ gần trọn chuỗi và thông tin hoàn toàn hợp lệ, nhưng cũng với kỳ vọng đóng góp nhỏ.

IV.5.3 Biên lợi nhuận âm

Cuối cùng chúng ta xem cột `cogs`, tức giá vốn hàng bán. Cột này tương quan rất mạnh với doanh thu, và chính vì mạnh như vậy mà nó đáng để xem kỹ.

Tương quan giá vốn và số ngày biên lợi nhuận âm

```

1 negative_margin = sales.cogs > revenue
2 quintile = pd.qcut(revenue, 5, labels=False)
3 share = (sales.assign(q=quintile, neg=negative_margin)
4          .groupby("q", observed=True).neg.mean() * 100)
5
6 print(f" Tương quan với doanh thu: {revenue.corr(sales.cogs):.4f}")
7 print(f" Ngày biên lợi nhuận âm: {int(negative_margin.sum())}")

```

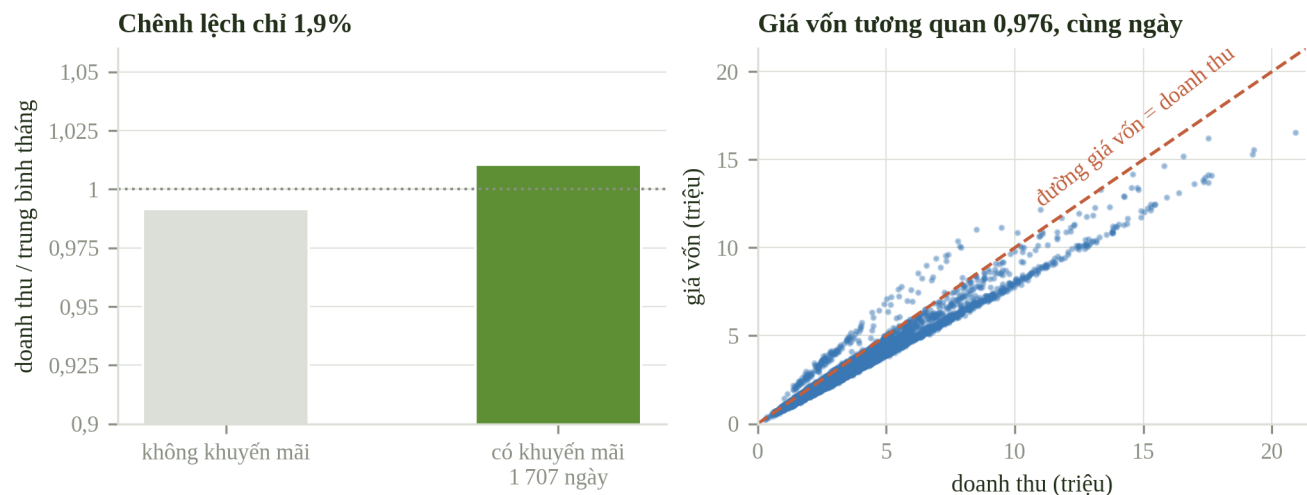
```

8         f" ({100 * negative_margin.mean():.1f}%)"
9     print("    Tỷ lệ theo nhóm năm phân vị doanh thu:")
10    print("        " + "        ".join(f"{v:.1f}%" for v in share))

```

Kết quả

Tương quan với doanh thu: 0.9760
 Ngày biên lợi nhuận âm: 382 (10.0%)
 Tỷ lệ theo nhóm năm phân vị doanh thu:
 21.4% 14.0% 7.4% 3.9% 3.1%



Hình 11: Chênh lệch doanh thu giữa ngày có và không có khuyến mãi, và quan hệ giữa giá vốn với doanh thu cùng ngày.

Khung phải của Hình 11 cho thấy có 382 ngày, tức 10% chuỗi, mà giá vốn vượt quá doanh thu. Tỷ lệ này giảm đều từ 21.4% ở nhóm doanh thu thấp nhất xuống 3.1% ở nhóm cao nhất, nên đây không phải nhiều ngẫu nhiên mà là đặc điểm của những ngày bán chậm. Chúng ta không sửa gì ở đây, vì biến mục tiêu là doanh thu chứ không phải lợi nhuận.

Điều đáng nói hơn nằm ở con số tương quan 0.976. Giá vốn gắn chặt với doanh thu đến mức chỉ cần đưa nó vào mô hình là điểm số sẽ đẹp lên rất nhiều. Nhưng giá vốn của ngày d chỉ biết được sau khi ngày d đã bán xong, tức tại thời điểm chạy dự báo trước 28 ngày thì con số ấy chưa tồn tại. Vì vậy cột `cogs` bị loại khỏi danh sách đặc trưng, và Mục V.5 sẽ cho thấy điểm số trông như thế nào nếu chúng ta để nó lọt vào.

IV.6 Tự tương quan theo độ trễ

Câu hỏi cuối của mục này là sau mốc $d - H$ thì còn lại bao nhiêu tín hiệu. Chúng ta trả lời bằng cách đo hệ số tự tương quan ở nhiều độ trễ, trên chuỗi đã khử miền năm để hai miền ở Mục IV.1 không làm sai lệch kết quả.

Hệ số tự tương quan theo từng độ trễ

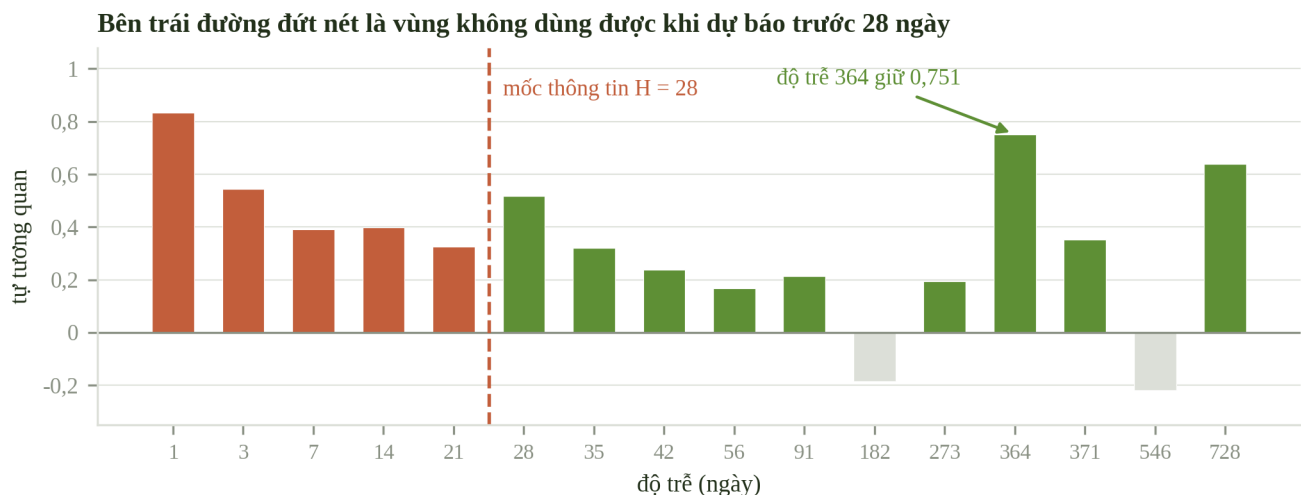
```

1 z = sales.year_ratio
2 lags = [1, 3, 7, 14, 21, 28, 35, 42, 56,
3         91, 182, 273, 364, 371, 546, 728]
4 acf = {lag: float(z.autocorr(lag)) for lag in lags}
5
6 acf_table = pd.DataFrame({"acf": acf})
7 acf_table["trạng thái"] = np.where(acf_table.index >= H,
8                                   "dùng được", "bị chặn")
9 print(acf_table.round(3).rename_axis("độ trễ").to_string())

```

Kết quả

độ trễ	acf	trạng thái
1	0.833	bị chặn
3	0.545	bị chặn
7	0.391	bị chặn
14	0.399	bị chặn
21	0.325	bị chặn
28	0.518	dùng được
35	0.321	dùng được
42	0.239	dùng được
56	0.167	dùng được
91	0.213	dùng được
182	-0.184	dùng được
273	0.194	dùng được
364	0.751	dùng được
371	0.353	dùng được
546	-0.220	dùng được
728	0.638	dùng được



Hình 12: Hệ số tự tương quan theo độ trễ. Trong vùng được phép, độ trễ 364 ngày giữ 0.751, gần bằng độ trễ 1 ngày vốn bị chặn.

Hình 12 chia bảng làm hai vùng. Độ trễ mạnh nhất toàn bảng là 1 ngày với hệ số 0.833, nhưng nó nằm bên trái mốc $H = 28$ nên không dùng được: tại thời điểm dự báo, chúng ta chưa biết doanh thu của hôm qua. Trong vùng được phép, độ trễ 364 ngày dẫn đầu với hệ số 0.751, gần bằng độ trễ 1 ngày.

Cùng kỳ năm trước mang gần như trọn vẹn lượng tín hiệu mà chúng ta mất khi phải lùi 28 ngày, nên nhóm đặc trưng lịch sử ở Mục V sẽ tập trung quanh mốc 364 ngày thay vì cố khai thác vùng gần.

Bảng 3 gom các quan sát của mục này cùng quyết định tương ứng.

Quan sát	Quyết định	Áp dụng ở
Mức doanh thu dứt gãy năm 2019	Tách riêng lần cắt kiểm định rơi vào năm đó	Mục VI
Phân phối lệch phải, độ lệch 1.669	Huấn luyện trên thang log, đo bằng WAPE	Mục VI
169 ngày cao tụ quanh chỗ giao tháng	Giữ nguyên, không cắt ngoại lai	Mục V
Chu kỳ ngày trong tháng, biên độ 2.84 lần	Thêm cột đếm ngược đến cuối tháng	Mục V
Hiệu ứng Tết phần lớn là hiệu ứng tháng	Giữ nhóm Tết với kỳ vọng đóng góp thấp	Mục V
Độ trễ 364 ngày mạnh nhất trong vùng hợp lệ	Nhóm lịch sử tập trung quanh mốc này	Mục V

Bảng 3: Mỗi quan sát ở mục này chốt một quyết định cụ thể, kèm mục sẽ áp dụng quyết định đó.

V. Xây dựng đặc trưng

Ở phần này chúng ta biến chuỗi 3,833 ngày thành một bảng đặc trưng để đưa vào mô hình cây. Toàn bộ công việc bị chi phối bởi một ràng buộc duy nhất, là mốc thông tin $d - H$ đã nêu ở đoạn Mở đầu. Vì vậy chúng ta bắt đầu bằng cách phát biểu ràng buộc đó cho thật rõ, rồi mới xây từng nhóm đặc trưng, sau đó kiểm tra tự động xem có cột nào lỗi vi phạm hay không.

Đọc tệp đã làm sạch

```
1 raw = pd.read_csv("sales_clean.csv", parse_dates=["date"])
2 raw = raw.sort_values("date").reset_index(drop=True)
3 print(f"Đọc {len(raw):,} dòng, khoảng dự báo H = {H} ngày")
```

Kết quả

Đọc 3,833 dòng, khoảng dự báo H = 28 ngày

V.1 Quy tắc lùi thời gian

Vùng dữ liệu được phép dùng khi chạy dự báo

ví dụ ngày 15 tháng 9 năm 2022; hai mươi bảy ngày ngay trước đó chưa tồn tại

Ngày	Doanh thu	Cách d	Đặc trưng dùng được
2021-09-16	1.853.200	-364	lag_364
2021-09-19	1.952.438	-361	lag364_smooth7
...	...	...	...
2022-06-16	4.143.877	-91	lag_91
2022-08-18	4.171.975	-28	lag_28
2022-08-19	2.011.109	-27	không có
...	...	...	...
2022-09-14	2.125.189	-1	không có
2022-09-15	?	d	cần dự báo

Được phép
cách ngày d ít nhất 28 ngày

Bị chặn
27 ngày, bảng chỉ trích ba dòng làm mẫu

Ô cần dự báo

Hình 13: Phân tích vùng dữ liệu được sử dụng khi chạy dự báo.

Khi dự báo cho ngày d , mô hình chỉ được biết dữ liệu đến hết ngày $d - H$. Hai mươi tám ngày ngay trước d chưa tồn tại ở thời điểm chạy dự báo, nên mọi đặc trưng dẫn xuất từ doanh thu đều phải lùi ít nhất H ngày. Hình 13 minh họa vùng được phép dùng bằng một bảng trích thật từ dữ liệu.

Quy tắc nghe thì đơn giản nhưng rất dễ vi phạm, vì cách viết trong pandas không nói cho chúng ta biết cửa sổ đang chạm tới ngày nào. Để thấy rõ điều đó, chúng ta so sánh năm cách viết cùng tính trung bình trượt 28 ngày.

Cách kiểm là phá hoại bằng cách nhân doanh thu của 27 ngày cuối chuỗi lên 9.3 lần rồi xem giá trị ở hàng cuối của mỗi cách viết có đổi theo hay không. Cách viết nào đổi theo tức là nó đang nhìn vào vùng lẽ ra bị chặn.

Phá hoại 27 ngày cuối để chấm năm cách viết trung bình trượt

```

1 def rolling_variants(y):
2     """Năm cách viết cùng tính trung bình trượt 28 ngày."""
3     return {
4         "y.rolling(28).mean()": y.rolling(28).mean(),
5         "y.shift(1).rolling(28).mean()": y.shift(1).rolling(28).mean(),
6         "y.shift(7).rolling(28).mean()": y.shift(7).rolling(28).mean(),
7         "y.rolling(28, center=True).mean().shift(28)":
8             y.rolling(28, center=True).mean().shift(28),
9         "y.shift(28).rolling(28).mean() (an toàn)":
10            y.shift(28).rolling(28).mean(),
11     }
12
13 broken = raw.copy()
14 broken.loc[broken.index[-(H - 1):], "revenue"] *= 9.3
15 intact = rolling_variants(raw["revenue"])
16 damaged = rolling_variants(broken["revenue"])
17
18 change = pd.Series({name: abs(float(intact[name].iloc[-1])
19                             - float(damaged[name].iloc[-1]))
20                     for name in intact})
21 verdict = np.where(change < 1e-9, "AN TOÀN", "RÒ RỈ")
22 print(pd.DataFrame({"phản ứng ở hàng cuối": change,
23                    "kết luận": verdict}))
24 .to_string(float_format="{:.3e}".format))

```

Kết quả

	phản ứng ở hàng cuối	kết luận
y.rolling(28).mean()	1.442e+07	RÒ RỈ
y.shift(1).rolling(28).mean()	1.371e+07	RÒ RỈ
y.shift(7).rolling(28).mean()	9.337e+06	RÒ RỈ
y.rolling(28, center=True).mean().shift(28)	5.110e+06	RÒ RỈ
y.shift(28).rolling(28).mean() (an toàn)	0.000e+00	AN TOÀN

Đánh giá các phương pháp tạo dữ liệu dạng chuỗi

Cách viết	Cửa sổ nhìn vào	Mới nhất	Mức thay đổi	độ lớn
<code>y.rolling(28).mean()</code>	<code>y[d-27:d]</code>	<code>d</code>	14,42 tr	
<code>y.shift(1).rolling(28).mean()</code>	<code>y[d-28:d-1]</code>	<code>d - 1</code>	13,71 tr	
<code>y.shift(7).rolling(28).mean()</code>	<code>y[d-34:d-7]</code>	<code>d - 7</code>	9,34 tr	
<code>y.rolling(28, center=True).mean().shift(28)</code>	<code>y[d-41:d-14]</code>	<code>d - 14</code>	5,11 tr	
<code>y.shift(28).rolling(28).mean()</code>	<code>y[d-55:d-28]</code>	<code>d - 28</code>	0	

phá hoại 28 nhân gần nhất rồi đo mức thay đổi của đặc trưng ở hàng cuối

Hình 14: Điểm mới nhất mà cửa sổ chạm tới quyết định tất cả: chỉ `shift(28).rolling(28)` dừng ở `d - 28` nên an toàn để sử dụng.

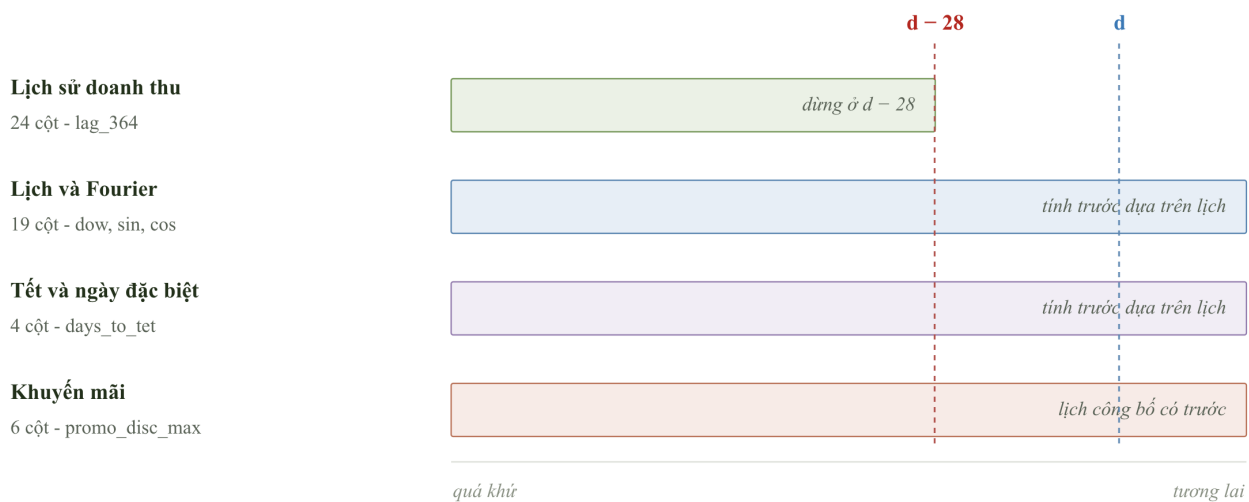
Theo Hình 14, bốn cách viết đầu đều phản ứng, chỉ cách cuối cùng đứng yên. Điều đáng chú ý là cách thứ tư, `rolling(28, center=True).mean().shift(28)`, trông rất giống cách an toàn vì cũng có `shift(28)` ở cuối, nhưng cửa sổ giữa tâm nhìn cả sang hai phía nên nó vẫn chạm tới ngày `d - 14`.

Từ đó chúng ta rút ra một quy tắc đọc mã: điều cần nhìn là **điểm mới nhất mà cửa sổ chạm tới**, và điểm đó phải nằm ở `d - H` hoặc xa hơn. Với cách viết an toàn, `shift(28)` đẩy toàn bộ chuỗi lùi 28 ngày trước, nên cửa sổ 28 ngày sau đó chỉ trải từ `d - 55` đến `d - 28`.

V.2 Các nhóm đặc trưng

Chúng ta chia 53 đặc trưng theo một tiêu chí duy nhất, là mỗi nhóm biết được đến ngày nào tại thời điểm chạy dự báo. Hình 15 vẽ lại tiêu chí đó.

53 đặc trưng chia thành bốn nhóm



Hình 15: 53 đặc trưng chia bốn nhóm. Chỉ nhóm lịch sử doanh thu bị mốc $d - 28$ chặn lại, ba nhóm còn lại biết được đến tận ngày d .

V.2.1 Nhóm lịch sử doanh thu

Đây là nhóm duy nhất bị mốc $d - H$ chặn lại, vì nó dẫn xuất trực tiếp từ biến mục tiêu. Nhóm này gồm độ trễ thô lấy bằng doanh thu của một ngày trong quá khứ, thống kê trượt tính trên nền đã lùi H ngày, và tỉ số cùng xu hướng giúp mô hình tách mức doanh thu ra khỏi hình dạng biến động.

Dựng 24 cột lịch sử trên nền đã lùi H ngày

[illegible]

```

21
22 # tỉ số và xu hướng: tách mức ra khỏi hình dạng
23 df["ratio_28_364"] = df.roll_mean_28 / df.roll_mean_364
24 df["ratio_91_364"] = df.roll_mean_91 / df.roll_mean_364
25 df["trend_28_91"] = df.roll_mean_28 - df.roll_mean_91
26 df["cv_91"] = df.roll_std_91 / df.roll_mean_91
27 return df

```

Danh sách độ trễ được chọn theo kết quả ở Mục IV.6. Mốc 28 là điểm gần nhất còn được phép, mốc 364 và 371 bám lấy cùng kỳ năm trước vốn giữ hệ số tự tương quan 0.751, còn các mốc ở giữa lấp khoảng trống để mô hình có thể nội suy.

Hai cột `lag364_smooth7` và `lag364_smooth28` là bản làm mượt của cùng kỳ năm trước. Chúng lùi lần lượt 361 và 350 ngày rồi lấy trung bình một cửa sổ ngắn, nên điểm mới nhất mà chúng chạm tới vẫn nằm quanh mốc 364 chứ không tiến gần *d*.

Lưu ý

Biến `safe` trong đoạn mã trên là một mẹo nhỏ nhưng quan trọng. Thay vì viết `y.rolling(w).mean().shift(H)` cho từng cửa sổ và phải nhớ đặt `shift` ở đúng chỗ, chúng ta lùi một lần duy nhất ngay từ đầu rồi mọi phép trượt sau đó đều an toàn theo. Nhờ vậy chúng ta chỉ phải nhớ một chỗ duy nhất.

V.2.2 Nhóm lịch

Nhóm lịch suy ra hoàn toàn từ ngày tháng nên không cần lùi. Với ngày *d* bất kỳ trong tương lai, chúng ta luôn biết nó là thứ mấy, ngày bao nhiêu và thuộc tháng nào.

Dựng cột lịch và bốn cặp Fourier

```

1 def add_calendar_features(df):
2     """Đặc trưng lịch, ba tầng mùa vụ đều biết trước nên không lùi."""
3     d = df["date"]
4     df["dow"] = d.dt.dayofweek
5     df["dom"] = d.dt.day
6     df["month"] = d.dt.month
7     df["quarter"] = d.dt.quarter
8     df["doy"] = d.dt.dayofyear
9     df["days_in_month"] = d.dt.days_in_month
10    df["dom_reverse"] = df.days_in_month - df.dom # 0 = cuối tháng
11    df["is_weekend"] = (df.dow >= 5).astype("int8")
12    df["is_eom"] = (df.dom_reverse <= 3).astype("int8")
13    df["is_som"] = (df.dom <= 2).astype("int8")
14    df["is_odd_year"] = (d.dt.year % 2).astype("int8")
15    for k in (1, 2, 3, 4):
16        angle = 2 * np.pi * k * df.doy / 365.25
17        df[f"fourier_sin{k}"] = np.sin(angle)
18        df[f"fourier_cos{k}"] = np.cos(angle)
19    return df

```

Cột `dom_reverse` chính là quyết định rút ra ở Mục IV.4. Nó đếm ngược đến cuối tháng, nên ngày cuối tháng luôn mang giá trị 0 dù tháng đó dài 28, 30 hay 31 ngày. Nếu chỉ có cột `dom` thì mô hình phải học riêng cho từng độ dài tháng, còn với `dom_reverse` thì quy luật cuối tháng gom về một chỗ.

Bốn cặp sin và cos là khai triển Fourier của chu kỳ năm. Chúng cho mô hình cây một cách mô tả mùa vụ tròn tru, thay vì phải cắt trục thời gian thành từng khoảng bằng các ngưỡng rời rạc.

V.2.3 Nhóm Tết và ngày đặc biệt

Tết Nguyên đán trượt theo lịch dương nên không suy ra được từ ngày tháng dương lịch, mà phải tra một bảng ngày đã biết trước. Bảng này phủ cả năm nằm ngoài phạm vi dữ liệu, để hai đầu chuỗi không bị tính sai khoảng cách.

Tra bảng Tết và đánh dấu ngày đặc biệt

```

1 RETAIL_SPECIAL_DAYS = [(1, 1), (3, 8), (4, 30), (5, 1), (9, 2),
2                        (10, 20), (11, 11), (12, 12), (12, 24), (12, 25)]
3
4 def add_tet_features(df):
5     """Đặc trưng Tết. Âm lịch dịch chuyển nên phải tra bảng."""
6     d = df["date"]
7     tet = pd.to_datetime(pd.Series(TET_DATES))
8     diff = np.stack([(d - t).dt.days.values for t in tet])
9     df["days_to_tet"] = diff[np.abs(diff).argmin(axis=0),
10                             np.arange(len(df))]
11     df["tet_before"] = df.days_to_tet.between(-30, -1).astype("int8")
12     df["tet_after"] = df.days_to_tet.between(0, 14).astype("int8")
13     sp = set(RETAIL_SPECIAL_DAYS)
14     df["is_special_day"] = np.array(
15         [(m, dd) in sp for m, dd in zip(d.dt.month, d.dt.day)],
16         dtype="int8")
17     return df

```

Cột `is_special_day` đánh dấu các mốc bán lẻ theo lịch dương. Ngoài ngày lễ chính thức, danh sách còn có 8 tháng 3, 20 tháng 10, 11 tháng 11 và 12 tháng 12 vì đây là những mốc mua sắm dù không phải ngày nghỉ.

Như đã thấy ở Mục IV.5.1, chúng ta giữ nhóm này với kỳ vọng đóng góp thấp. Bốn cột là đủ để mô hình nắm được hiệu ứng nếu có, mà không tốn nhiều chỗ nếu không có.

V.2.4 Nhóm khuyến mãi

Lịch khuyến mãi được công bố trước, nên tại thời điểm chạy dự báo chúng ta đã biết ngày d nằm trong chương trình nào. Đây là nhóm duy nhất ngoài lịch mà biết được đến tận ngày d .

Quét lịch khuyến mãi và xử lý chương trình chồng nhau

```

1 def add_promo_features(df):
2     """Lịch khuyến mãi công bố trước nên biết được tới ngày d."""
3     pr = pd.read_csv("promotions.csv",
4                      parse_dates=["start_date", "end_date"])
5     df["promo_n"] = 0
6     df["promo_disc_max"] = 0.0
7     df["promo_day_idx"] = -1
8     df["promo_streetwear"] = 0
9     df["promo_outdoor"] = 0
10    for _, p in pr.iterrows():
11        m = (df.date >= p.start_date) & (df.date <= p.end_date)
12        if not m.any():
13            continue
14        df.loc[m, "promo_n"] += 1
15        df.loc[m, "promo_disc_max"] = np.maximum(
16            df.loc[m, "promo_disc_max"], float(p.discount_value))
17        # hai chương trình có thể chồng nhau: giữ chương trình mới
18        # bắt đầu, nếu không kết quả phụ thuộc thứ tự dòng trong CSV
19        idx = (df.loc[m, "date"] - p.start_date).dt.days
20        cur = df.loc[m, "promo_day_idx"]
21        df.loc[m, "promo_day_idx"] = np.where(cur < 0, idx,
22                                              np.minimum(cur, idx))
23        if p.applicable_category == "Streetwear":
24            df.loc[m, "promo_streetwear"] = 1
25        elif p.applicable_category == "Outdoor":
26            df.loc[m, "promo_outdoor"] = 1
27    df["promo_any"] = (df.promo_n > 0).astype("int8")
28    return df

```

Chi tiết đáng chú ý ở đây là cách xử lý khi hai chương trình chồng lên nhau. Cột `promo_day_idx` đếm số ngày kể từ lúc chương trình bắt đầu, và nếu một ngày thuộc hai chương trình thì chúng ta lấy giá trị nhỏ hơn, tức chương trình mới bắt đầu gần đây nhất. Nếu không quy định rõ như vậy, kết quả sẽ phụ thuộc vào thứ tự dòng trong tệp CSV, và chỉ cần ai đó sắp xếp lại tệp là mọi con số phía sau đổi theo.

Ghép bốn nhóm

Bốn hàm trên chạy nối tiếp nhau trên cùng một bảng, rồi chúng ta liệt kê danh sách đặc trưng và kiểm rằng không cột nào lọt ra ngoài bốn nhóm.

Chạy nối tiếp bốn hàm rồi đối chiếu tổng số cột

```

1 df = add_promo_features(add_tet_features(add_calendar_features(
2     add_lag_features(raw.copy()))))
3 skip_columns = ("date", TARGET, "cogs")
4 features = [c for c in df.columns if c not in skip_columns]
5
6 GROUPS = {

```

```

7     "lịch sử doanh thu": ("lag", "roll", "ratio", "trend", "cv_"),
8     "lịch": ("dow", "dom", "month", "quarter", "doy", "days_in",
9             "is_week", "is_eom", "is_som", "is_odd", "fourier"),
10    "Tết và ngày đặc biệt": ("days_to_tet", "tet_", "is_special"),
11    "khuyến mãi": ("promo",),
12 }
13 groups = {}
14 for name, prefixes in GROUPS.items():
15     columns = [c for c in features if c.startswith(prefixes)]
16     groups[name] = columns
17     print(f" {name:24s} {len(columns):3d} cột"
18           f" {'', '.join(columns[:4])} ...")
19 print(f" {'TỔNG':24s} {len(features):3d} cột")
20 assert sum(len(v) for v in groups.values()) == len(features)

```

Kết quả

lịch sử doanh thu	24 cột	lag_28, lag_35, lag_42, lag_56 ...
lịch	19 cột	dow, dom, month, quarter ...
Tết và ngày đặc biệt	4 cột	days_to_tet, tet_before, tet_after, is_special_day ...
khuyến mãi	6 cột	promo_n, promo_disc_max, promo_day_idx,
promo_streetwear ...		
TỔNG	53 cột	

Câu `assert` ở cuối lại là một chốt chặn, ai thêm cột mới mà quên đưa tiền tố vào `GROUPS` thì chương trình dừng ngay, thay vì để cột đó biến mất khỏi mọi phép phân tích theo nhóm ở Mục VIII.

V.3 Kiểm tra rò rỉ dữ liệu

Bốn hàm ở Mục V.2 viết bằng tay, mà quy tắc lùi thời gian thì rất dễ vi phạm như Mục V.1 đã cho thấy. Vì vậy chúng ta cần một phép kiểm tự động chạy được mỗi lần sửa mã, chứ không thể chỉ đọc lại bằng mắt. Rò rỉ dữ liệu là một lỗi phổ biến và khó phát hiện, được phân loại đầy đủ trong **Leakage in Data Mining (Kaufman, 2012)**.

Ý tưởng vẫn là phá hoại, nhưng lần này áp cho toàn bộ nhóm lịch sử. Tại hàng dự báo cho ngày d , nếu chúng ta thay đổi nhân trong vùng từ $d - H + 1$ đến d thì không đặc trưng nào ở hàng d được phép đổi theo.

Phép thử phá hoại chạy ở nhiều mốc thời gian

```

1 def leakage_probe(raw, n_checks=8):
2     """Phép thử phá hoại.
3
4     Đổi nhân trong vùng bị chặn rồi xem đặc trưng có động không. Chạy ở
5     nhiều mốc chứ không chỉ hàng cuối, một lỗi chỉ lộ ra khi đã có đủ
6     lịch sử sẽ hoàn toàn im lặng nếu chỉ kiểm một hàng.
7     """
8     fa = add_lag_features(raw.copy())

```

```

9     cols = [c for c in fa.columns
10              if c.startswith(("lag", "roll", "ratio", "trend", "cv_"))]
11     rows = np.linspace(max(H, 400), len(raw) - 1, n_checks, dtype=int)
12     worst = 0.0
13     for row in rows:
14         b = raw.copy()
15         b.loc[b.index[row - H + 1: row + 1], "revenue"] *= 9.3
16         fb = add_lag_features(b)
17         diff = (fa[cols].iloc[row].fillna(-1)
18                 - fb[cols].iloc[row].fillna(-1)).abs().max()
19         worst = max(worst, float(diff))
20     return worst, len(rows), len(cols)
21
22
23 worst, n_rows_checked, n_columns = leakage_probe(raw)
24 print(f" Phá hoại {H} nhãn gần nhất tại {n_rows_checked} mốc,"
25       f" kiểm {n_columns} cột")
26 print(f" Thay đổi lớn nhất quan sát được = {worst:.3e}")
27 assert worst < 1e-9, "RỒ RỈ: có đặc trưng nhìn thấy nhãn bị chặn"

```

Kết quả

Phá hoại 28 nhãn gần nhất tại 8 mốc, kiểm 24 cột
Thay đổi lớn nhất quan sát được = 0.000e+00

Thay đổi lớn nhất bằng đúng 0 trên cả 24 cột và cả 8 mốc, nên nhóm lịch sử không rõ ràng. Phép thử chạy ở tám mốc rải đều vì nhiều cột chỉ có giá trị khi đã tích đủ lịch sử, nên một lỗi ở `roll_mean_364` sẽ im lặng nếu chúng ta chỉ kiểm một hàng đầu chuỗi.

V.4 Giai đoạn khởi động

Cuối cùng chúng ta bỏ những ngày đầu chuỗi không đủ dữ liệu để tính đặc trưng. Cột `lag_371` cần 371 ngày lịch sử mới có giá trị, còn `roll_mean_364` cần cửa sổ 364 ngày trên nền đã lùi 28 ngày.

Loại các ngày chưa đủ lịch sử rồi ghi tệp đặc trưng

```

1 n_before = len(df)
2 df = df.dropna(subset=["lag_371", "roll_mean_364"])
3 df = df.reset_index(drop=True)
4 dropped_pct = 100 * (n_before - len(df)) / n_before
5
6 print(f" Bỏ {n_before - len(df)} ngày đầu chuỗi ({dropped_pct:.1f}%)")
7 print(f" Còn {len(df):,} ngày, từ {df.date.min().date()}
8       f" đến {df.date.max().date()}")
9 df.to_csv(f"features_h{H}.csv", index=False)
10 save_results("04_features", {"features": features,
11                             "n_features": len(features),
12                             "warmup": {"dropped": n_before - len(df),

```

```
13 "remaining": len(df)}})
```

Kết quả

Bỏ 371 ngày đầu chuỗi (9.7%)
Còn 3,462 ngày, từ 2013-07-10 đến 2022-12-31

Sử dụng 371 ngày khởi động để tính đặc trưng lag_364

mất 371 ngày đầu để có đặc trưng cùng kỳ năm trước

04/07/2012

31/12/2022



3.462 ngày dùng được

371 ngày khởi động, 9.7% chuỗi, hết ngày 09/07/2013

Hình 16: Bỏ 371 ngày đầu chuỗi, tức 9.7%, để đổi lấy nhóm đặc trưng cùng kỳ năm trước.

Theo Hình 16, chúng ta mất 371 ngày, tức 9.7% chuỗi, và đổi lại là đặc trưng cùng kỳ năm trước, mốc mạnh nhất trong vùng được phép dùng theo Mục IV.6.

Cần lưu ý rằng phần bị bỏ nằm ở đầu chuỗi, tức toàn bộ năm 2012 và nửa đầu 2013. Đây là giai đoạn có miền doanh thu cao, thuộc về thời kỳ trước cú đứt gãy ở Mục IV.1. Bỏ nó đi cũng đồng nghĩa với việc mô hình được huấn luyện trên phần dữ liệu gần với hiện tại hơn.

Tập `features_h28.csv` thu về có 3,462 dòng và 53 đặc trưng. Đây là đầu vào cho Mục VI.

V.5 Điểm số khi để rò rỉ lọt vào

Phép thử trên xác nhận chúng ta không bị rò rỉ. Nhưng để thấy vì sao phải cẩn thận đến vậy, chúng ta thử cố tình để rò rỉ lọt vào rồi xem điểm số đổi ra sao. Cả hai lần đều đọc `features_h28.csv` vừa ghi ở Mục V.4, huấn luyện trên phần train của lần cắt TEST và đo trên nửa cuối năm 2022. Cột dùng để minh họa là `cogs`, giá vốn của chính ngày cần dự báo, vốn có tương quan 0.976 với doanh thu như đã thấy ở Mục IV.5.3.

Huấn luyện hai lần chỉ khác nhau một cột

```
1 ready = pd.read_csv(f"features_h{H}.csv", parse_dates=["date"])
2 train, test = split_by_date(ready, *TEST_FOLD[1:])
3 y_train_log = np.log1p(train.revenue.values)
4 y_test = test.revenue.values
5
6 runs = {f"{len(features)} đặc trưng hợp lệ": features,
7         "thêm cogs cùng ngày (rò rỉ)": features + ["cogs"]}
8 scores, models = {}, {}
```



```

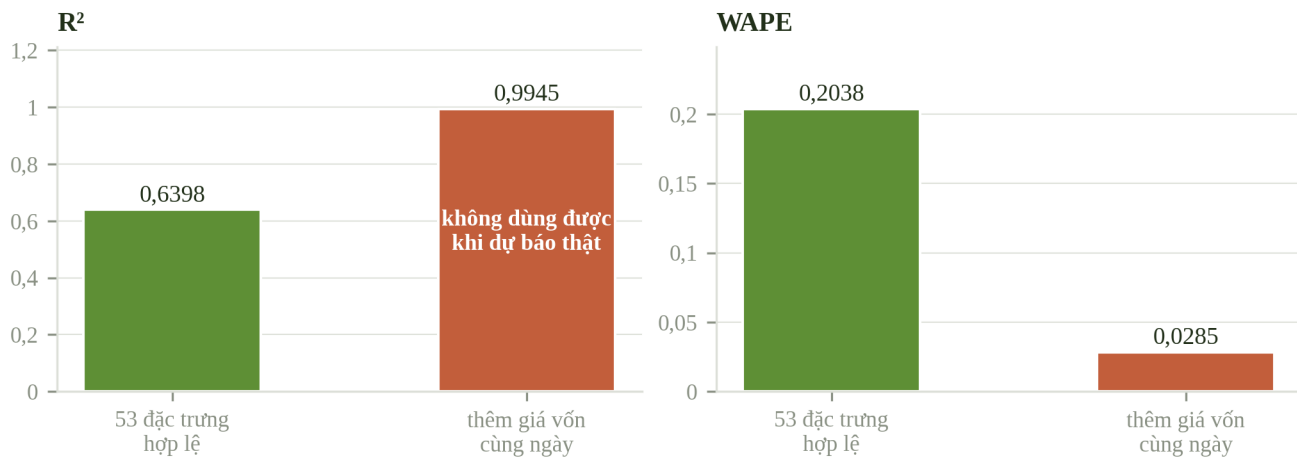
9 for name, columns in runs.items():
10     models[name] = get_models()["LightGBM"]
11     models[name].fit(train[columns], y_train_log)
12     prediction = np.expml(models[name].predict(test[columns]))
13     scores[name] = metrics(y_test, prediction)
14
15 print(pd.DataFrame(scores).T.round(4).to_string())
16 importance = pd.Series(models["thêm cogs cùng ngày (rò ri)"]
17                        .feature_importances_,
18                        index=runs["thêm cogs cùng ngày (rò ri)"])
19 print("\n Năm cột đứng đầu bảng quan trọng: "
20       + ", ".join(f"{k}={v}" for k, v
21                   in importance.nlargest(5).items()))

```

Kết quả

	WAPE	bias	R2
53 đặc trưng hợp lệ	0.2038	-0.1181	0.6398
thêm cogs cùng ngày (rò ri)	0.0285	0.0102	0.9945

Năm cột đứng đầu bảng quan trọng: cogs=2817, lag_364=1316, lag_28=1092, lag364_smooth7=809, lag_371=759



Hình 17: Cùng một mô hình, chạy với 53 đặc trưng hợp lệ và khi thêm giá vốn cùng ngày. Con số đẹp lên hẳn nhưng không dùng được khi dự báo thật.

Hình 17 đặt hai điểm số cạnh nhau. Chỉ cần thêm một cột, R^2 nhảy từ 0.6398 lên 0.9945 và WAPE giảm từ 0.2038 xuống 0.0285, tức sai số nhỏ đi bảy lần. Trong bảng xếp hạng độ quan trọng, cogs đứng đầu với 2.817 điểm, hơn gấp đôi cột thứ hai là lag_364 với 1.316.

Con số ấy không có ý nghĩa trong thực tế: như Mục IV.5.3 đã nêu, mô hình đang sử dụng trực tiếp thông tin về biến mục tiêu thay vì thực hiện dự báo. Đây cũng là một dấu hiệu cần xem xét: khi R^2 gần 1 trong một bài toán dự báo, chúng ta nên kiểm tra xem có đặc trưng nào đứng đầu bảng mức quan trọng một cách bất thường hay không.

VI. Xây dựng mô hình

Ở phần này chúng ta huấn luyện mô hình trên bảng đặc trưng vừa xây. Trước hết chúng ta quyết định cách chia dữ liệu, độ đo dùng để chấm điểm và mốc so sánh, vì ba thứ đó chi phối cách đọc mọi con số phía sau. Bốn lần cắt và hai hàm chấm điểm đã khai ở khối chuẩn bị chung tại Mục I. Ở đây chúng ta giải thích vì sao chúng có hình dạng như vậy, và vì sao `CV_FOLDS` chỉ liệt kê Fold 2 với Fold 3.

Đọc bảng đặc trưng đã xây ở Mục V

```
1 df = pd.read_csv(f"features_h{H}.csv", parse_dates=["date"])
2 features = load_results("04_features")["features"]
3 series = df.set_index("date").revenue
```

Hai hàm chấm điểm dùng lại suốt mục này

```
1 def evaluate_fold(fold, columns, model=None):
2     """Huấn luyện trên phần train, đo trên phần kiểm định."""
3     train, valid = split_by_date(df, fold[1], fold[2], fold[3])
4     if model is None:
5         model = get_models()["LightGBM"]
6         model.fit(train[columns], np.log1p(train.revenue.values))
7         prediction = np.expm1(model.predict(valid[columns]))
8         return metrics(valid.revenue.values, prediction)
9
10 def cross_validate(columns, model_name="LightGBM"):
11     """Chạy qua ba lần cắt, trung bình hai lần ổn định."""
12     scores = {}
13     for fold in FOLDS:
14         model = get_models()[model_name]
15         scores[fold[0]] = evaluate_fold(fold, columns, model)["WAPE"]
16     scores["CV"] = float(np.mean([scores[f] for f in CV_FOLDS]))
17     return scores
```

VI.1 Chia tập theo thời gian

Với dữ liệu chuỗi thời gian, chúng ta không được chia ngẫu nhiên. Chia ngẫu nhiên sẽ đặt những ngày tương lai vào tập huấn luyện và những ngày quá khứ vào tập kiểm định, tức mô hình được học đáp án của chính khoảng thời gian nó sắp phải dự báo.

Cách chia phù hợp là cắt theo mốc thời gian **Bergmeir (2018)**. Ở đây, chúng ta sử dụng cửa sổ mở rộng với ba lần cắt, mỗi lần lùi thêm một năm, cùng một tập kiểm tra cuối cùng dành riêng cho Mục VII.

Một số bảng ở mục này có cột TEST, nhưng cột đó chỉ để đối chiếu và không có quyết định nào của tài liệu dựa vào nó.

Dựng bốn lần cắt và đo khoảng đệm

```

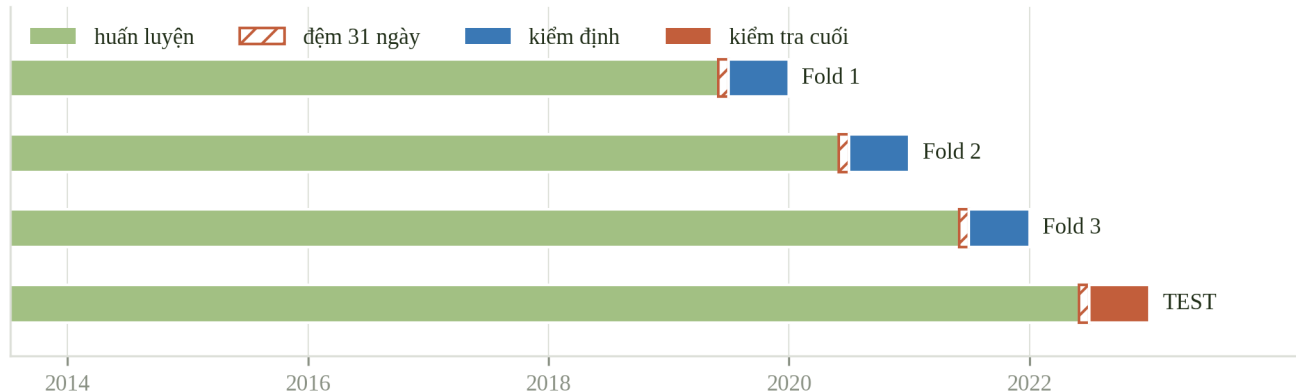
1 folds = []
2 for name, train_end, valid_start, valid_end in FOLDS + [TEST_FOLD]:
3     train, valid = split_by_date(df, train_end, valid_start, valid_end)
4     buffer_days = (pd.Timestamp(valid_start)
5                   - pd.Timestamp(train_end)).days
6     folds.append({"name": name, "train_end": train_end,
7                  "valid": f"{valid_start} đến {valid_end}",
8                  "n_train": len(train), "n_valid": len(valid),
9                  "buffer_days": buffer_days})
10 print(pd.DataFrame(folds).to_string(index=False))

```

Kết quả

	name	train_end	valid	n_train	n_valid	buffer_days
Fold 1	2019-05-31	2019-07-01 đến	2019-12-31	2152	184	31
Fold 2	2020-05-31	2020-07-01 đến	2020-12-31	2518	184	31
Fold 3	2021-05-31	2021-07-01 đến	2021-12-31	2883	184	31
TEST	2022-05-31	2022-07-01 đến	2022-12-31	3248	184	31

Gốc trượt theo thời gian, mỗi lần cắt lùi thêm một năm



Hình 18: Bốn lần cắt theo cửa sổ mở rộng. Khoảng đệm 31 ngày giữa tập huấn luyện và tập kiểm định dài hơn $H = 28$, nên loại bỏ hoàn toàn vùng chồng lấn.

Đáng chú ý nhất trong Hình 18 là cột **buffer_days**, giữa tập huấn luyện và tập kiểm định chúng ta để trống 31 ngày, và khoảng đệm này bắt buộc phải có vì hai tập dùng nhãn theo hai cách khác nhau.

Một hàng huấn luyện ở ngày T mang nhãn là doanh thu của chính ngày T . Trong khi đó, hàng kiểm định đầu tiên là một hàng dự báo, nên tại thời điểm chạy nó chỉ được biết dữ liệu tới $T - H$. Nếu hai tập sát nhau thì mô hình đã nhìn thấy nhãn của những ngày mà lẽ ra nó chưa được biết. Đệm 31 ngày dài hơn $H = 28$ nên khoảng chồng lấn đó biến mất hoàn toàn.

Lưu ý

Lần cắt thứ nhất rơi vào nửa cuối năm 2019, đúng chỗ mức doanh thu đứt gãy như Mục IV.1 đã chỉ ra. Mô hình huấn luyện trên miền cao rồi đem đo trên miền thấp, nên sai số của lần cắt này phản ánh cú đứt gãy chứ không phản ánh chất lượng mô hình. Vì vậy chúng ta vẫn chạy Fold 1 và vẫn báo cáo con số của nó, nhưng điểm kiểm định chéo chỉ lấy trung bình Fold 2 và Fold 3.

VI.2 Chọn độ đo

Độ đo chính của tài liệu là WAPE, tức tổng sai số tuyệt đối chia cho tổng giá trị thật:

$$\text{WAPE} = \frac{\sum_d |y_d - \hat{y}_d|}{\sum_d y_d}. \quad (2)$$

Lý do chọn WAPE nằm ở hai tính chất của bộ dữ liệu đã tìm được ở Mục IV. Thứ nhất, phân phối lệch phải với vài ngày cực trị, nên độ đo phải bền với những ngày đó. Thứ hai, khoảng giá trị chênh nhau 75 lần, nên độ đo không được nhạy với những ngày doanh thu thấp.

Hai độ đo quen thuộc khác đều vướng một trong hai tính chất trên. RMSE bình phương sai số nên một ngày lệch gấp đôi sẽ đóng góp gấp bốn, và với 169 ngày cực trị thì con số cuối cùng chủ yếu phản ánh vài ngày đó. MAPE chia sai số cho giá trị thật của từng ngày, nên một ngày doanh thu 280 nghìn lệch 100 nghìn sẽ bị tính nặng hơn hẳn một ngày doanh thu 20 triệu lệch 2 triệu, dù về mặt kinh doanh thì ngược lại.

WAPE tránh được cả hai vì nó cộng sai số tuyệt đối rồi chia cho tổng, tức mỗi đồng doanh thu có trọng số như nhau. Một cách đọc trực quan là WAPE bằng 0.20 nghĩa là tổng sai số chiếm 20% tổng doanh thu.

Nhưng WAPE có một điểm mù, là nó lấy trị tuyệt đối nên không phân biệt dự báo cao hơn thực tế với dự báo thấp hơn thực tế. Một mô hình luôn dự báo thấp 15% và một mô hình lúc cao lúc thấp có thể cho cùng một WAPE, trong khi hệ quả kinh doanh của chúng khác hẳn nhau. Vì vậy chúng ta luôn báo cáo WAPE kèm thiên lệch:

$$\text{bias} = \frac{\bar{\hat{y}} - \bar{y}}{\bar{y}}. \quad (3)$$

Thiên lệch dương nghĩa là mô hình dự báo cao hơn thực tế, âm là thấp hơn. Mục VII.2 sẽ cho thấy con số này quan trọng thế nào với mô hình cuối cùng.

Một số bảng còn kèm cột R^2 , phần phương sai của chuỗi thật mà mô hình giải thích được. Chúng ta không dùng nó để chọn mô hình, nhưng vẫn in ra vì R^2 gần 1 trong một bài toán dự báo là dấu hiệu đáng ngờ, như Mục V.5 đã cho thấy.

VI.3 Mô hình cơ sở

Tiếp theo chúng ta dựng một mốc so sánh, vì không có mốc thì con số WAPE 0.24 chẳng cho biết được điều gì cả, nó có thể là rất tốt, cũng có thể là tệ hơn cả phép dự báo bằng tay đơn

giản nhất.

Chúng ta dựng bảy mô hình cơ sở, đều là những quy tắc thẳng thừng không cần huấn luyện. Điểm chung của cả bảy là chúng đều tôn trọng mốc $d - H$.

Bảy mô hình cơ sở

```

1 level_factor = (series.shift(H).rolling(91).mean()
2               / series.shift(364 + H).rolling(91).mean())
3 BASELINES = {
4     "y[d-28]": series.shift(H),
5     "trung bình 7 ngày, lùi 28": series.shift(H).rolling(7).mean(),
6     "trung bình 28 ngày, lùi 28": series.shift(H).rolling(28).mean(),
7     "cùng kỳ năm trước y[d-364]": series.shift(364),
8     "cùng kỳ hai năm trước y[d-728]": series.shift(728),
9     "trung bình hai năm cùng kỳ":
10         (series.shift(364) + series.shift(728)) / 2,
11     "cùng kỳ năm trước nhân hệ số mức":
12         series.shift(364) * level_factor,
13 }
```

Ba mô hình đầu khai thác vùng gần tức lấy doanh thu quanh mốc $d - 28$, ba mô hình tiếp theo khai thác cùng kỳ năm trước theo đúng gợi ý của Mục IV.6, còn mô hình cuối cùng kết hợp cả hai bằng cách lấy cùng kỳ năm trước rồi nhân với một hệ số điều chỉnh miền để bù cho cú đứt gãy ở Mục IV.1.

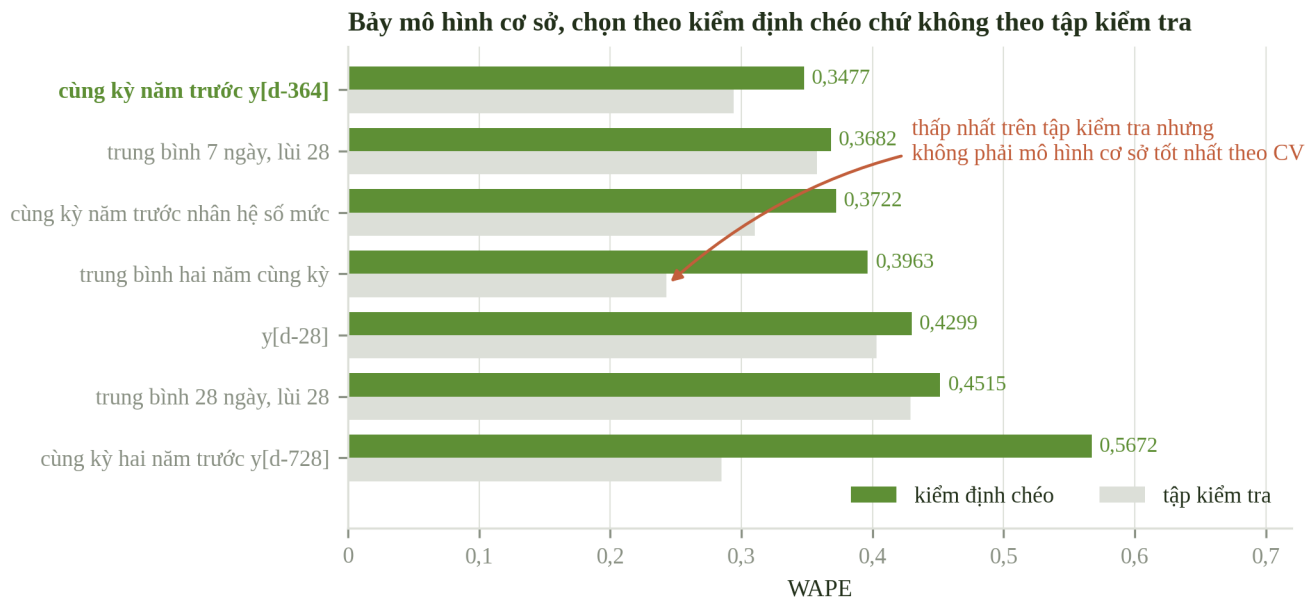
Đo bảy mô hình cơ sở trên mọi lần cắt

```

1 rows = []
2 for name, prediction in BASELINES.items():
3     row = {"baseline": name}
4     for fold in FOLDS + [TEST_FOLD]:
5         _, valid = split_by_date(df, fold[1], fold[2], fold[3])
6         p = np.nan_to_num(prediction.reindex(valid.date).values)
7         row[fold[0]] = metrics(valid.revenue.values, p)["WAPE"]
8     row["CV"] = float(np.mean([row[f] for f in CV_FOLDS]))
9     rows.append(row)
10
11 baseline_table = pd.DataFrame(rows).set_index("baseline")
12 columns_shown = ["Fold 1", "Fold 2", "Fold 3", "CV", "TEST"]
13 print(baseline_table[columns_shown].round(4).rename_axis(
14       "mô hình cơ sở").to_string())
15
16 best = baseline_table["CV"].idxmin()
17 chosen = {"name": best,
18          "WAPE_cv": float(baseline_table.loc[best, "CV"]),
19          "WAPE_test": float(baseline_table.loc[best, "TEST"])}
```

Kết quả

mô hình cơ sở	Fold 1	Fold 2	Fold 3	CV	TEST
y[d-28]	0.4330	0.4080	0.4519	0.4299	0.4031
trung bình 7 ngày, lùi 28	0.3856	0.3705	0.3660	0.3682	0.3574
trung bình 28 ngày, lùi 28	0.4557	0.4558	0.4473	0.4515	0.4290
cùng kỳ năm trước y[d-364]	0.6892	0.3362	0.3592	0.3477	0.2941
cùng kỳ hai năm trước y[d-728]	0.7597	0.7235	0.4109	0.5672	0.2847
trung bình hai năm cùng kỳ	0.7052	0.4517	0.3410	0.3963	0.2428
cùng kỳ năm trước nhân hệ số mức	0.2872	0.3592	0.3853	0.3722	0.3105



Hình 19: Bảy mô hình cơ sở. Cùng kỳ năm trước tốt nhất theo kiểm định chéo với WAPE 0.3477, còn mô hình thấp nhất trên tập kiểm tra chỉ xếp thứ tư theo kiểm định chéo.

Theo Hình 19, nếu chọn theo kết quả kiểm định chéo thì mô hình cơ sở tốt nhất là phép dự báo bằng cùng kỳ năm trước, với WAPE 0.3477. Chúng ta lấy con số này làm mốc cho toàn bộ phần còn lại của tài liệu.

Bên cạnh đó, kết quả vừa in ra cũng cho thấy một cái bẫy đáng nhớ. Nếu chọn theo cột TEST thì mô hình tốt nhất lại là trung bình hai năm cùng kỳ với WAPE 0.2428, thấp hơn hẳn. Nhưng chính mô hình đó chỉ đạt 0.3963 trên kiểm định chéo, tức xếp thứ tư trong bảy. Nó thắng ở cột TEST vì nửa cuối năm 2022 tình cờ hợp với cách tính đó, và đây là đặc điểm của một cửa sổ 184 ngày chứ không phải một quy luật.

Lưu ý

Vì vậy chúng ta chốt mọi lựa chọn bằng kiểm định chéo, và chỉ mở tập kiểm tra một lần ở cuối. Nếu vừa nhìn cột TEST vừa chọn thì tập kiểm tra không còn là một phép đo độc lập nữa, mà đã trở thành một phần của quá trình huấn luyện.

VI.4 Huấn luyện các mô hình cây

Chúng ta huấn luyện năm mô hình cây của Module 03, xếp theo độ phức tạp tăng dần: cây quyết định, rừng ngẫu nhiên, Gradient Boosting của scikit-learn, XGBoost **Chen (2016)** và LightGBM **Ke (2017)**. Cả năm chạy trên cùng bộ 53 đặc trưng, cùng hạt giống ngẫu nhiên, và cùng ba lần cắt.

Chạy năm mô hình trên ba lần cắt

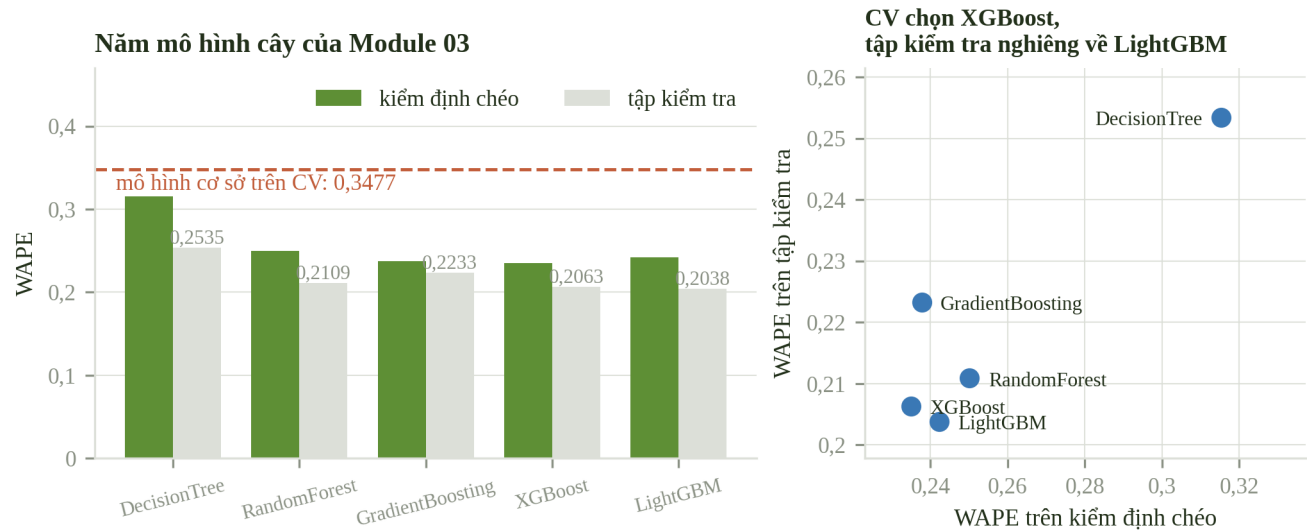
```

1 records = []
2 for fold in FOLDS:
3     for model_name, model in get_models().items():
4         started = time.time()
5         score = evaluate_fold(fold, features, model)
6         records.append(dict(fold=fold[0], model=model_name,
7                             fit_seconds=time.time() - started, **score))
8 cv_table = pd.DataFrame(records)
9
10 stable = (cv_table[cv_table.fold.isin(CV_FOLDS)].groupby("model")
11             .agg(WAPE=("WAPE", "mean"), bias=("bias", "mean"),
12                 fit_seconds=("fit_seconds", "mean"))
13             .sort_values("WAPE"))
14 fold1 = cv_table[cv_table.fold == "Fold 1"].set_index("model")
15 summary = stable.assign(fold1_WAPE=fold1.WAPE, fold1_bias=fold1.bias)
16 print(summary.round(4).rename_axis("mô hình").to_string())

```

Kết quả

mô hình	WAPE	bias	fit_seconds	fold1_WAPE	fold1_bias
XGBoost	0.2350	-0.0536	1.5425	0.5154	0.4934
GradientBoosting	0.2377	-0.0819	9.5813	0.4751	0.4333
LightGBM	0.2423	-0.0627	1.2777	0.4842	0.4781
RandomForest	0.2502	-0.0656	8.5873	0.5190	0.5017
DecisionTree	0.3153	-0.0899	0.0583	0.6504	0.6155



Hình 20: Cả năm mô hình cây đều vượt ngưỡng cơ sở 0.3477. Kiểm định chéo chọn XGBoost, còn tập kiểm tra nghiêng về LightGBM.

Theo Hình 20, XGBoost dẫn đầu với WAPE 0.2350, và cả năm mô hình đều vượt mô hình cơ sở 0.3477. Khoảng cách giữa mô hình tốt nhất và mô hình kém nhất là 0.2350 so với 0.3153, tức chênh nhau khoảng một phần ba.

Trong kết quả vừa in ra, trước hết chúng ta đọc cột `fit_seconds`, LightGBM chạy 1.3 giây trong khi Gradient Boosting của scikit-learn mất 9.6 giây, nhanh gấp 7.5 lần dù cho kết quả tương đương. Chênh lệch này đến từ thuật toán histogram, vốn gom giá trị liên tục thành các thùng rời rạc trước khi tìm ngưỡng cắt **Ke (2017)**. Riêng cột `fit_seconds` phụ thuộc vào máy chạy, nên chạy lại trên máy khác sẽ ra con số khác. Thứ giữ nguyên là tỉ lệ giữa các mô hình.

Thứ hai là cột `bias`, cả năm mô hình đều mang thiên lệch âm, tức đều dự báo thấp hơn thực tế. Đây là dấu hiệu của một nguyên nhân chung chứ không phải đặc điểm riêng của mô hình nào, và Mục VII.2 sẽ truy tìm nguyên nhân đó.

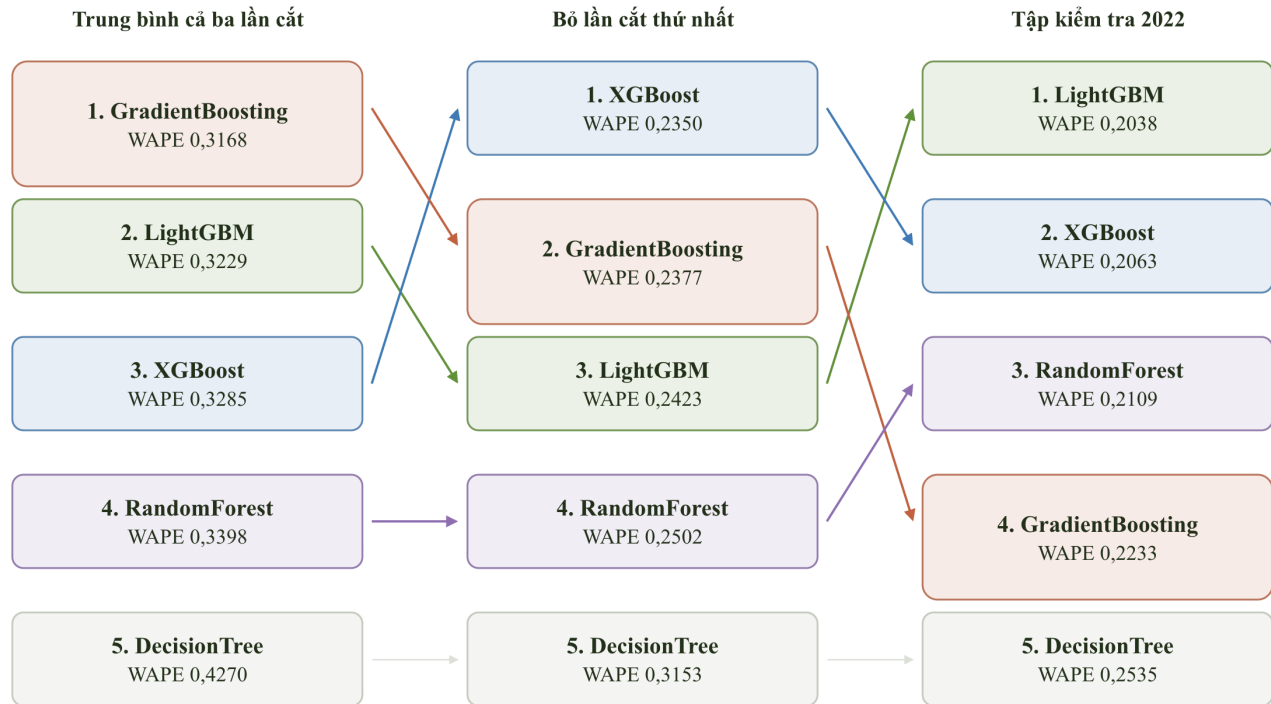
Thứ ba là hai cột của Fold 1, WAPE nhảy lên 0.4751 đến 0.6504 và thiên lệch đổi dấu thành dương, tức mô hình dự báo cao hơn thực tế, đúng như Mục VI.1 đã dự đoán.

Thứ hạng không ổn định

Tuy nhiên thứ hạng vừa thấy phụ thuộc vào cách chúng ta gộp các lần cắt, và Hình 21 đặt ba cách đọc cạnh nhau.

Đánh giá mô hình dựa trên các phương pháp khác nhau

cùng bộ đặc trưng, cùng hạt giống, chỉ đổi cách chia dữ liệu



Hình 21: So sánh các mô hình theo cách đánh giá: gộp cả ba lần cắt, bỏ Fold 1, hay đo trên tập kiểm tra 2022.

Nếu lấy trung bình cả ba lần cắt thì Gradient Boosting đứng đầu, nếu bỏ Fold 1 thì XGBoost đứng đầu, còn nếu đo trên tập kiểm tra thì LightGBM đứng đầu. Ba cách đọc cho ba câu trả lời khác nhau, mà chênh lệch giữa các mô hình thì rất nhỏ: LightGBM và XGBoost cách nhau 0.0025 WAPE trên một cửa sổ 184 ngày.

Theo kết quả kiểm định chéo, XGBoost là mô hình được chọn để đánh giá. Tài liệu sử dụng LightGBM riêng cho phần minh họa cách giải thích mô hình ở Mục VIII; lựa chọn này phục vụ mục đích trình bày và không thay đổi kết quả lựa chọn mô hình bằng kiểm định chéo.

VI.5 Đóng góp của từng nhóm đặc trưng

Cuối cùng chúng ta xem 53 đặc trưng có thật sự cần thiết không, bằng cách thêm dần từng nhóm rồi đo lại sau mỗi bước, bắt đầu từ bộ tối giản chỉ có hai cột.

Chạy năm bộ đặc trưng thêm dần trên ba lần cắt

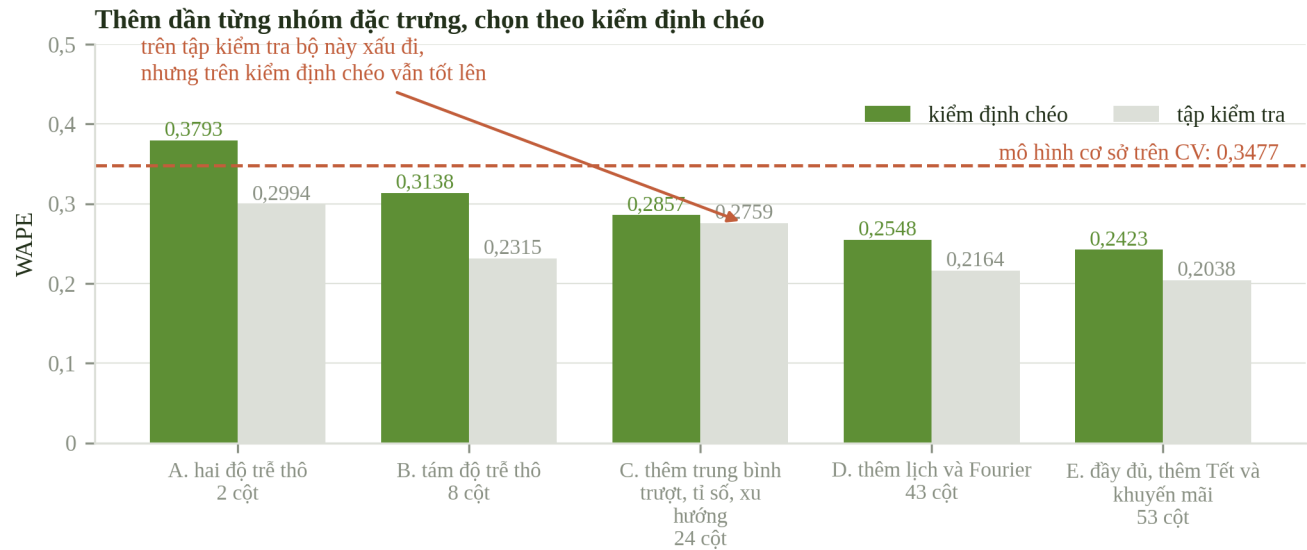
```

1 HISTORY = ("lag", "roll", "ratio", "trend", "cv_")
2 CALENDAR = ("dow", "dom", "month", "quarter", "doy", "days_in",
3             "is_week", "is_eom", "is_som", "is_odd", "fourier")
4 FEATURE_SETS = {
5     "A. hai độ trễ thô": ["lag_28", "lag_364"],
6     "B. tám độ trễ thô": [c for c in features if c.startswith("lag_")],
7     "C. thêm trung bình trượt, tỉ số, xu hướng":
8         [c for c in features if c.startswith(HISTORY)],
9     "D. thêm lịch và Fourier":
10        [c for c in features if c.startswith(HISTORY + CALENDAR)],
11     "E. đầy đủ, thêm Tết và khuyến mãi": features,
12 }
13
14 rows = []
15 for name, columns in FEATURE_SETS.items():
16     scores = cross_validate(columns)
17     scores["TEST"] = evaluate_fold(TEST_FOLD, columns)["WAPE"]
18     rows.append({"set": name, "n_columns": len(columns), **scores})
19
20 add_table = pd.DataFrame(rows).set_index("set")
21 base_cv = add_table["CV"].iloc[0]
22 add_table["vs_A_pct"] = 100 * (add_table.CV / base_cv - 1)
23 shown = {"n_columns": "cột", "Fold 2": "Fold 2", "Fold 3": "Fold 3",
24          "CV": "CV", "vs_A_pct": "so với A %", "TEST": "TEST"}
25 print(add_table[list(shown)].round(4).rename(columns=shown)
26        .rename_axis("bộ đặc trưng").to_string())
27 save_results("05_models", {"baselines": baseline_table.to_dict("index"),
28                             "baseline_chosen": chosen,
29                             "cv": stable.to_dict("index"),
30                             "feature_sets": add_table.to_dict("index")})

```

Kết quả

bộ đặc trưng	cột	Fold 2	Fold 3	CV	so với A %	TEST
A. hai độ trễ thô	2	0.3581	0.4004	0.3793	0.0000	0.2994
B. tám độ trễ thô	8	0.3136	0.3139	0.3138	-17.2710	0.2315
C. thêm trung bình trượt, tỉ số, xu hướng	24	0.2970	0.2744	0.2857	-24.6616	0.2759
D. thêm lịch và Fourier	43	0.2657	0.2440	0.2548	-32.8090	0.2164
E. đầy đủ, thêm Tết và khuyến mãi	53	0.2631	0.2214	0.2423	-36.1230	0.2038



Hình 22: Thêm dần từng nhóm, WAPE kiểm định chéo giảm từ 0.3793 xuống 0.2423. Riêng ở bộ C, tập kiểm tra xấu đi trong khi kiểm định chéo vẫn tốt lên.

Theo Bảng 4 và Hình 22, WAPE trên kiểm định chéo giảm đều từ 0.3793 xuống 0.2423, tổng cộng 36.1%. Bước nhảy lớn nhất là từ hai lên tám độ trễ, giảm 17.3% chỉ với sáu cột thêm vào, và bộ B cũng là bộ đầu tiên vượt mô hình cơ sở, trong khi bộ A còn tệ hơn mức đó 9.1%.

Cột TEST lại kể một câu chuyện khác. Ở bộ C, WAPE trên tập kiểm tra tăng từ 0.2315 lên 0.2759, tức xấu đi, trong khi trên kiểm định chéo nó vẫn giảm từ 0.3138 xuống 0.2857. Hai con số mâu thuẫn nhau, và chúng ta tin con số kiểm định chéo vì nó đo trên hai cửa sổ độc lập thay vì một. Đây cũng là một minh họa nữa cho cái bẫy đã gặp ở Mục VI.3: một cửa sổ 184 ngày không đủ để phân biệt cải thiện thật với may mắn.

Nhóm thêm vào	Số cột	CV	So với bộ liền trước	Nhận xét
A. Hai độ trễ thô	2	0.3793	—	Bộ tối giản, vẫn tệ hơn mô hình cơ sở 0.3477
B. Tám độ trễ thô	8	0.3138	−17.3%	Bước nhảy lớn nhất, và là bộ đầu tiên vượt mô hình cơ sở
C. Trung bình trượt và tỉ số	24	0.2857	−8.9%	Thống kê trượt bù thêm cho độ trễ thô
D. Lịch và Fourier	43	0.2548	−10.8%	Nhóm đóng góp lớn thứ hai
E. Tết và khuyến mãi	53	0.2423	−4.9%	Đúng kỳ vọng thấp đặt ra ở Mục IV.5

Bảng 4: Đóng góp của từng nhóm đặc trưng, đo bằng WAPE trên kiểm định chéo. Cột áp chót là mức giảm so với bộ liền trước, không phải so với bộ A.

VII. Đánh giá trên tập kiểm tra

Đến đây chúng ta đã chốt xong mọi lựa chọn bằng kiểm định chéo gồm bộ đặc trưng, cách chia tập, độ đo và mô hình cơ sở. Phần này đo trên tập kiểm tra, và con số thu về là ước lượng trung thực nhất về chất lượng mô hình khi chạy thật.

Tập kiểm tra gồm 184 ngày của nửa cuối năm 2022, từ 1 tháng 7 đến 31 tháng 12. Nó có xuất hiện ở vài bảng của các mục trước để đối chiếu, nhưng chưa từng quyết định điều gì.

VII.1 Kết quả dự báo

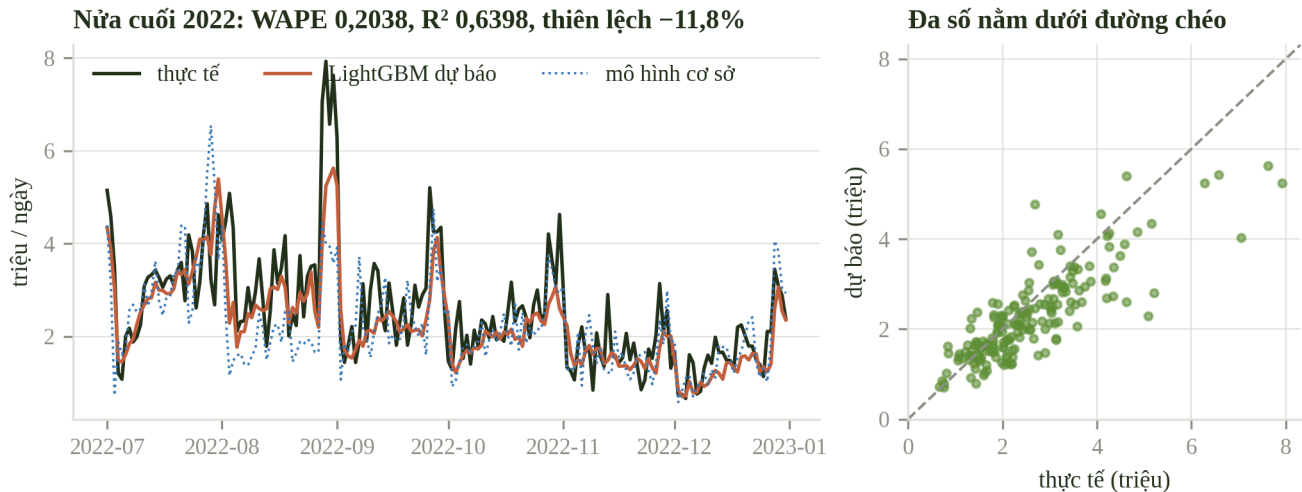
Chúng ta huấn luyện lại cả năm mô hình trên toàn bộ phần dữ liệu trước mốc chia, rồi đo trên tập kiểm tra.

Đo năm mô hình trên tập kiểm tra

```
1 train, test = split_by_date(df, *TEST_FOLD[1:])
2 y_train_log = np.log1p(train.revenue.values)
3 y_test = test.revenue.values
4
5 scores = {}
6 for model_name, model in get_models().items():
7     model.fit(train[features], y_train_log)
8     prediction = np.expml(model.predict(test[features]))
9     scores[model_name] = metrics(y_test, prediction)
10
11 table = pd.DataFrame(scores).T.sort_values("WAPE")
12 baseline = load_results("05_models")["baseline_chosen"]
13 baseline_wape = baseline["WAPE_test"]
14 table["vs_baseline_pct"] = 100 * (table.WAPE / baseline_wape - 1)
15 print(table.round(4).to_string())
16 print(f"\n mô hình cơ sở    WAPE {baseline['WAPE_test']:.4f}")
```

Kết quả

	WAPE	bias	R2	vs_baseline_pct
LightGBM	0.2038	-0.1181	0.6398	-30.6943
XGBoost	0.2063	-0.1316	0.6588	-29.8417
RandomForest	0.2109	-0.1008	0.6180	-28.2796
GradientBoosting	0.2233	-0.1621	0.5839	-24.0733
DecisionTree	0.2535	-0.0749	0.4731	-13.8087
mô hình cơ sở	WAPE 0.2941			



Hình 23: Dự báo của LightGBM và của mô hình cơ sở so với thực tế trên 184 ngày kiểm tra. Ở khung phải, đa số điểm nằm dưới đường chéo, tức mô hình dự báo thấp.

Hình 23 đặt đường dự báo cạnh đường thực tế. LightGBM đạt WAPE 0.2038, tốt hơn mô hình cơ sở 30.7%. Nói cách khác, tổng sai số chiếm khoảng 20% tổng doanh thu của nửa cuối 2022, trong khi phép dự báo bằng cùng kỳ năm trước để lại sai số 29%.

Cả năm mô hình đều vượt mô hình cơ sở, kém nhất là cây quyết định với 13.8%. Điều này xác nhận rằng bộ 53 đặc trưng thật sự mang thông tin, chứ không phải mô hình chỉ đang học lại chính cùng kỳ năm trước.

Mô hình chốt theo kiểm định chéo ở Mục VI.4 là XGBoost, còn tốt nhất trên tập kiểm tra lại là LightGBM, chênh nhau 0.0025 WAPE. Chúng ta không đổi lựa chọn sau khi nhìn thấy tập kiểm tra, vì làm vậy là phá đúng quy tắc đã đặt ra ở Mục VI.3. Tài liệu dùng LightGBM cho phần giải thích ở Mục VIII là một quyết định về trình bày, không phải kết quả của phép so sánh này.

VII.2 Thiên lệch của mô hình

Ở phần này chúng ta xem cột **bias**. Cả năm mô hình đều mang thiên lệch âm, và LightGBM dự báo thấp hơn thực tế 11.8%. Đây không phải nhiều ngẫu nhiên mà là một khuynh hướng có hệ thống, nên chúng ta cần truy ra nguyên nhân.

Giả thuyết đầu tiên là do phép đổi thang. Chúng ta huấn luyện trên $\log(1 + y)$ rồi dùng $\exp$ để đổi ngược, mà trung bình của hàm mũ không bằng hàm mũ của trung bình, nên phép đổi này vốn dĩ tạo ra thiên lệch âm. Cách chữa quen thuộc là nhân dự báo với hệ số Duan **Duan (1983)**, tính từ phần dư của tập huấn luyện.

Kiểm hai giả thuyết đầu trên thang log

```

1 model = get_models()["LightGBM"]
2 model.fit(train[features], y_train_log)
3 residual_log = np.log1p(y_test) - model.predict(test[features])
4 duan_factor = float(np.mean(np.exp(
5     y_train_log - model.predict(train[features]))))
6
7 mean_residual = residual_log.mean()
8 print(f" Trung bình phần dư trên thang log: {mean_residual:+.4f},"
9       f" tương đương {100 * (np.exp(mean_residual) - 1):+.1f}%")
10 print(f" Hệ số Duan từ phần dư train      : {duan_factor:.4f},"
11        f" chỉ chữa được {100 * (duan_factor - 1):+.2f}%")

```

Kết quả

Trung bình phần dư trên thang log: +0.1058, tương đương +11.2%
 Hệ số Duan từ phần dư train : 1.0016, chỉ chữa được +0.16%

Hệ số Duan bằng 1.0016 nên nó chỉ nâng dự báo lên 0.16%, quá nhỏ so với mức lệch 11.8% cần giải thích. Giả thuyết thứ nhất bị loại.

Con số thứ hai trong đoạn mã trên trả lời luôn giả thuyết thứ hai. Trung bình phần dư đo ngay trên thang log đã là +0.1058, tương đương +11.2%, tức gần trọn vẹn mức lệch cần giải thích. Điều đó có nghĩa là thiên lệch đã tồn tại từ trước khi đổi thang, nên nguyên nhân nằm ở chính mô hình chứ không nằm ở phép biến đổi.

Giả thuyết thứ ba là do miền doanh thu năm 2022 hồi phục. Dự báo cho nửa cuối 2022 chủ yếu dựa vào cùng kỳ năm trước, tức nửa cuối 2021, mà 2021 lại là năm miền thấp nhất chuỗi theo bảng ở Mục IV.1. Nếu nửa cuối 2022 cao hơn giai đoạn đó thì mô hình sẽ kéo dự báo về phía miền cũ.

So sánh miền nửa cuối 2022 với các năm trước

```

1 raw = pd.read_csv("sales_clean.csv", parse_dates=["date"])
2
3 def second_half_mean(year):
4     w = (raw.date.dt.year == year) & (raw.date.dt.month >= 7)
5     return float(raw.loc[w, "revenue"].mean())
6
7 recent_mean = np.mean([second_half_mean(y) for y in (2019, 2020, 2021)])
8 vs_2021 = 100 * (second_half_mean(2022) / second_half_mean(2021) - 1)
9 vs_recent = 100 * (second_half_mean(2022) / recent_mean - 1)
10 print(f" Nửa cuối 2022 so với cùng kỳ 2021      : {vs_2021:+.1f}%")
11 print(f" Nửa cuối 2022 so với trung bình 2019-2021: {vs_recent:+.1f}%")

```

Kết quả

Nửa cuối 2022 so với cùng kỳ 2021 : +16.2%
 Nửa cuối 2022 so với trung bình 2019-2021: +10.2%

Nửa cuối 2022 cao hơn cùng kỳ 2021 tới 16.2%, và cao hơn trung bình ba năm gần nhất 10.2%. Con số 10.2% này rất gần với mức thiên lệch 11.8% cần giải thích, nên giả thuyết thứ ba đứng vững.

Thiên lệch đến từ dữ liệu huấn luyện chứ không phải từ lỗi kỹ thuật, và Mục VIII.2 sẽ cho thấy cơ chế này ở mức từng ngày. Đây là hạn chế cố hữu của mọi mô hình học từ lịch sử, và cách chữa đúng không phải là nhân thêm một hệ số mà là huấn luyện lại định kỳ để mô hình bám theo miền mới.

VIII. Giải thích mô hình bằng SHAP

Mô hình đạt WAPE 0.2038 nhưng chúng ta chưa biết nó dựa vào đâu để cho ra con số đó. Ở phần này chúng ta dùng SHAP để trả lời hai câu hỏi là trên toàn bộ tập kiểm tra thì đặc trưng nào quan trọng nhất, và với một ngày cụ thể thì mô hình đã cộng trừ những gì.

SHAP **Lundberg (2017)** gán cho mỗi đặc trưng ở mỗi hàng một con số, là phần đóng góp của đặc trưng đó vào dự báo của hàng ấy. Với mô hình cây, thuật toán TreeSHAP **Lundberg (2020)** tính được giá trị này một cách chính xác thay vì xấp xỉ. Tính chất quan trọng nhất của nó là tính cộng tính với mức nền cộng tổng tất cả đóng góp phải bằng đúng dự báo của mô hình, không xê dịch.

Tính SHAP trên tập kiểm tra

```
1 import shap
2
3 model = get_models()["LightGBM"]
4 model.fit(train[features], y_train_log)
5 prediction_log = model.predict(test[features])
6
7 explainer = shap.TreeExplainer(model)
8 shap_values = explainer.shap_values(test[features])
9 base_value = float(np.ravel(explainer.expected_value)[0])
```

VIII.1 Mức quan trọng của từng đặc trưng

Để đo mức quan trọng trên toàn tập, chúng ta lấy trung bình trị tuyệt đối của SHAP theo từng cột. Dùng trị tuyệt đối vì một đặc trưng có thể đẩy dự báo lên ở ngày này và kéo xuống ở ngày khác, mà cả hai đều là đóng góp.

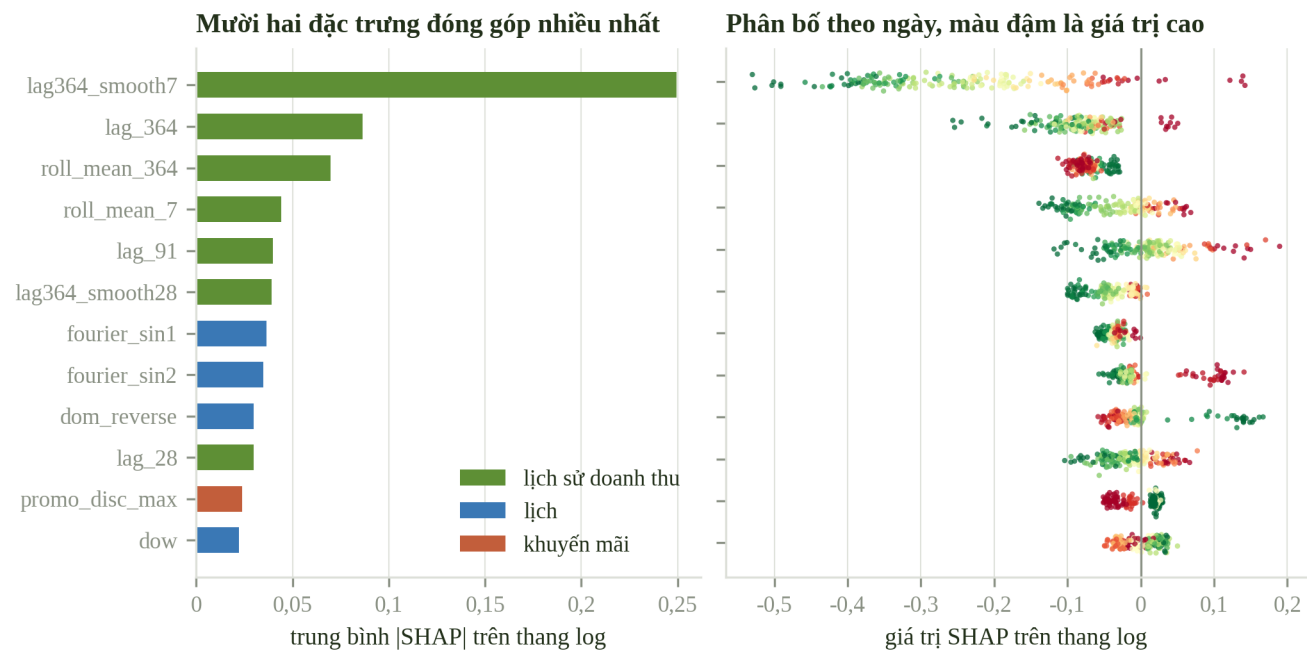
Xếp hạng đặc trưng theo trung bình trị tuyệt đối SHAP

```
1 mean_abs = pd.Series(np.abs(shap_values).mean(axis=0), index=features)
2 total_abs = float(mean_abs.sum())
3 ranking = mean_abs.sort_values(ascending=False)
4
5 top12 = ranking.head(12).to_frame("TB |SHAP|")
6 top12["tỉ trọng %"] = 100 * top12["TB |SHAP|"] / total_abs
7 print(top12.round(4).to_string())
```

Kết quả

	TB SHAP	tỉ trọng %
lag364_smooth7	0.2495	25.3978
lag_364	0.0863	8.7868
roll_mean_364	0.0696	7.0894

roll_mean_7	0.0441	4.4920
lag_91	0.0398	4.0500
lag364_smooth28	0.0390	3.9730
fourier_sin1	0.0364	3.7010
fourier_sin2	0.0348	3.5436
dom_reverse	0.0300	3.0508
lag_28	0.0297	3.0200
promo_disc_max	0.0238	2.4224
dow	0.0221	2.2514



Hình 24: Mười hai đặc trưng đóng góp nhiều nhất: bốn trong sáu vị trí đầu đều xoay quanh mốc 364 ngày.

Theo Hình 24, cột `lag364_smooth7` đứng đầu với tỉ trọng 25.4%, gấp gần ba lần cột thứ hai. Bốn trong sáu cột đầu bảng đều xoay quanh mốc 364 ngày, nên kết quả này khớp đúng với dự đoán rút ra từ Mục IV.6 là cùng kỳ năm trước là nguồn tín hiệu hợp lệ mạnh nhất, và mô hình dựa vào nó nhiều nhất.

Cột `dom_reverse` cũng lọt vào nhóm mười cột đầu bảng, nên quan sát về chu kỳ ngày trong tháng ở Mục IV.4 đã chuyển thành giá trị thật cho mô hình.

Gộp theo bốn nhóm đặc trưng

Tiếp theo chúng ta gộp theo nhóm. Xếp hạng từng cột chưa đủ để trả lời câu hỏi nhóm nào quan trọng hơn, vì một nhóm có nhiều cột sẽ tự nhiên chiếm tổng lớn hơn, nên chúng ta báo cáo cả tổng lẫn trung bình mỗi cột.

Đóng góp gộp theo bốn nhóm

```

1 group_shap = {}
2 for group_name, columns in groups.items():
3     total = float(mean_abs[columns].sum())
4     group_shap[group_name] = {"total": total,
5                               "share_pct": 100 * total / total_abs,
6                               "n_columns": len(columns),
7                               "mean_per_column": total / len(columns)}
8 print(pd.DataFrame(group_shap).T.round(4)
9       .astype({"n_columns": int}).to_string())

```

Kết quả

	total	share_pct	n_columns	mean_per_column
lịch sử doanh thu	0.7195	73.2448	24	0.0300
lịch	0.2086	21.2402	19	0.0110
Tết và ngày đặc biệt	0.0152	1.5488	4	0.0038
khuyến mãi	0.0390	3.9662	6	0.0065

Nhóm lịch sử doanh thu chiếm 73.2% tổng mức đóng góp. Nhóm này dẫn đầu theo cả hai cách đo: có tổng mức đóng góp lớn nhất và mức trung bình trên mỗi cột cao nhất, đạt 0.0300. Nhóm lịch đứng thứ hai với 21.2%.

Hai nhóm cuối đóng góp rất ít, đúng như kỳ vọng đặt ra từ Mục IV.5.1 và Mục IV.5.2. Nhóm Tết chỉ chiếm 1.5%, khớp với kết luận rằng phần lớn hiệu ứng Tết thật ra là hiệu ứng tháng, mà hiệu ứng tháng thì nhóm lịch đã nắm rồi. Nhóm khuyến mãi chiếm 4.0%, cũng khớp với chênh lệch 1.9% đã đo được.

Thứ tự hai nhóm đầu trùng với kết quả của phép thêm dần ở Mục VI.5. Riêng thứ tự giữa Tết và khuyến mãi thì chỉ SHAP tách được, vì phép thêm dần gộp cả hai vào chung bộ E. Sự trùng khớp ở hai nhóm đầu là một tín hiệu nhất quán, nhưng chưa phải xác nhận độc lập, vì cả hai phép đo đều chạy trên cùng bộ dữ liệu và cùng cửa sổ thời gian.

Kiểm tính cộng tính

Trước khi tin vào các con số ở trên, chúng ta kiểm lại tính cộng tính đã nêu ở đầu mục.

Kiểm tính cộng tính của ma trận SHAP

```

1 reconstructed = base_value + shap_values.sum(axis=1)
2 max_error = float(np.abs(reconstructed - prediction_log).max())
3 print(f" Ma trận SHAP {shap_values.shape}, "
4       f" sai số lớn nhất {max_error:.3e}")
5 print(f" Ngưỡng 1e-08: {'ĐẠT' if max_error < 1e-8 else 'HÔNG'}")

```

Kết quả

Ma trận SHAP (184, 53), sai số lớn nhất $4.796e-14$
 Ngưỡng $1e-08$: ĐẠT

Sai số lớn nhất là 4.8×10^{-14} , tức đúng ở mức sai số làm tròn của số thực. Ma trận SHAP cộng lại đúng bằng dự báo trên cả 184 ngày, nên chúng ta có thể đọc từng con số trong đó như một phép cộng thật.

VIII.2 Giải thích một dự báo cụ thể

Phần trước cho biết mô hình nhìn chung dựa vào đâu. Phần này đi vào một ngày cụ thể để xem nó cộng trừ những gì. Chúng ta chọn ngày 29 tháng 8 năm 2022, một ngày thứ Hai mà mô hình dự báo thấp hơn thực tế khá nhiều, vì ngày như vậy cho thấy rõ cả đóng góp dương lẫn âm.

Đóng góp của từng cột trong một ngày

```

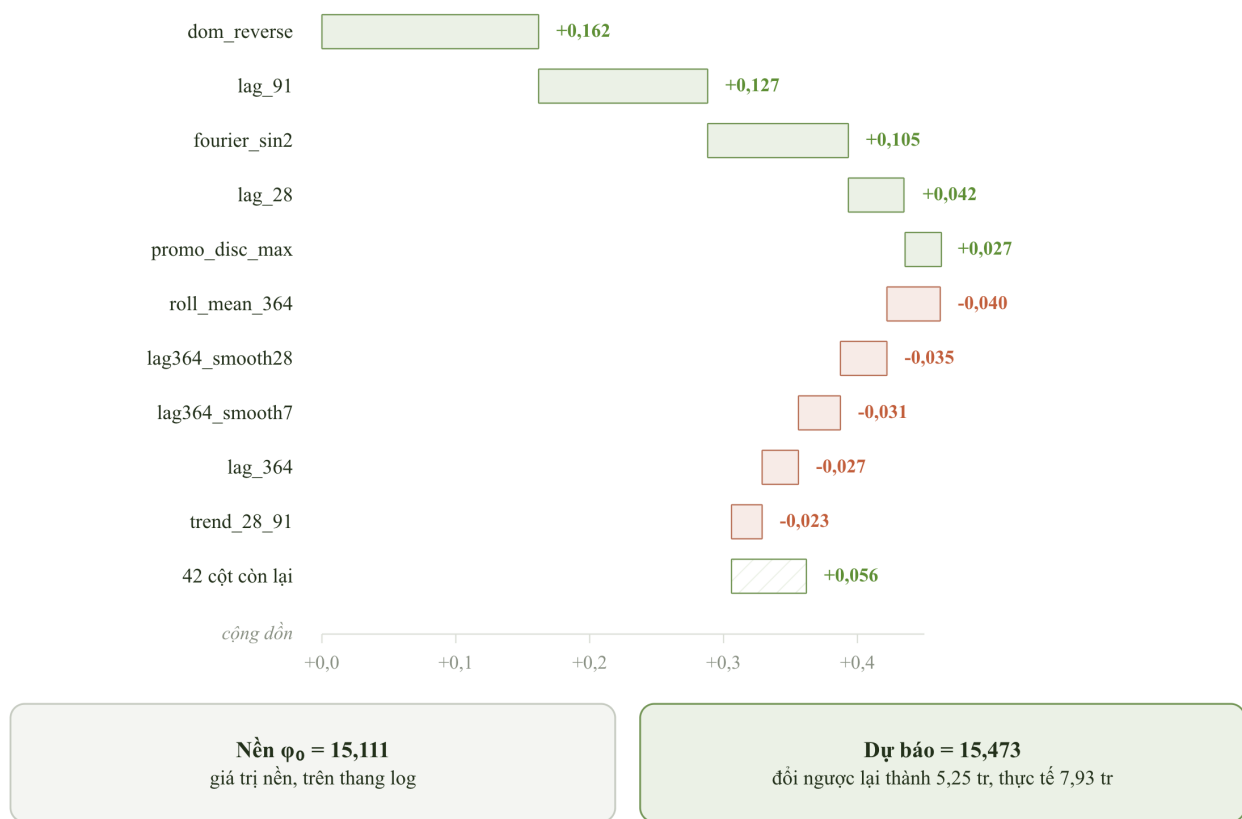
1 target_date = "2022-08-29"
2 index = int(np.where(test.date.values
3                 == np.datetime64(target_date))[0][0])
4 contribution = pd.Series(shap_values[index], index=features)
5 predicted = float(np.exp1(base_value + shap_values[index].sum()))
6 actual = float(y_test[index])
7
8 TOP_N = 5
9 push_up = contribution[contribution > 0].sort_values(ascending=False)
10 pull_down = contribution[contribution < 0].sort_values()
11 top = pd.concat([push_up.head(TOP_N), pull_down.head(TOP_N)])
12 top = top.to_frame("SHAP")
13 top["giá trị của cột"] = [f"{test[c].values[index]:.2f}"
14                           for c in top.index]
15 print(top.round(4).to_string())
16 print(f" {len(push_up) - TOP_N} cột đẩy lên còn lại"
17       f" {push_up[TOP_N:].sum():+.4f}")
18 print(f" {len(pull_down) - TOP_N} cột kéo xuống còn lại"
19       f" {pull_down[TOP_N:].sum():+.4f}")

```

Kết quả

	SHAP	giá trị của cột
dom_reverse	0.1618	2.00
lag_91	0.1266	8,158,502.00
fourier_sin2	0.1051	0.91
lag_28	0.0418	4,081,818.17
promo_disc_max	0.0269	0.00
roll_mean_364	-0.0402	3,012,303.23
lag364_smooth28	-0.0350	2,358,267.73
lag364_smooth7	-0.0312	3,315,017.81
lag_364	-0.0271	3,996,164.30
trend_28_91	-0.0230	-898,767.06
23 cột đẩy lên còn lại +0.1549		
19 cột kéo xuống còn lại -0.0986		

Giải thích mô hình với SHAP cho ngày 29 tháng 8 năm 2022



Hình 25: Từ mức nền 15.111, các đóng góp dương và âm cộng lại thành 15.473, đổi ngược thang log ra 5.25 triệu trong khi thực tế là 7.93 triệu.

Hình 25 vẽ phép cộng dồn này. Cột đẩy lên mạnh nhất là **dom_reverse** với giá trị 2, tức ngày 29 tháng 8 cách cuối tháng đúng hai ngày. Mô hình đã học được rằng cụm ngày cuối tháng có doanh thu cao, đúng như quan sát ở Mục IV.3, nên nó đẩy dự báo lên. Cột thứ hai là **lag_91** với giá trị 8.16 triệu, một mức cao so với trung bình, nên nó cũng đẩy lên.

Ngược lại, bốn trong năm cột kéo xuống mạnh nhất đều xoay quanh cùng kỳ năm trước. Cả bốn đều mang giá trị quanh 2.4 đến 4.0 triệu, trong khi thực tế ngày đó là 7.93 triệu, nên chúng kéo dự báo xuống. Đây chính là cơ chế tạo ra thiên lệch mà chúng ta đã truy ra ở Mục VII.2, nhìn thấy được ở mức từng ngày.

Cộng dồn về đúng dự báo

```

1 print(f" Nền {base_value:.4f} cộng đẩy lên {push_up.sum():+.4f}")
2     f" cộng kéo xuống {pull_down.sum():+.4f}")
3 print(f" bằng {base_value + contribution.sum():.4f} trên thang log,"
4     f" đổi ngược lại thành {predicted:,.0f}")
5 print(f" Thực tế ngày đó là {actual:,.0f},"
6     f" mô hình dự báo thấp hơn {100 * (1 - predicted / actual):.1f}%")

```

Kết quả

Nền 15.1109 cộng đẩy lên +0.6171 cộng kéo xuống -0.2551
bằng 15.4729 trên thang log, đổi ngược lại thành 5,245,517
Thực tế ngày đó là 7,928,797, mô hình dự báo thấp hơn 33.8%

Kết quả phép cộng khớp chính xác với giá trị dự báo. Mức dự báo 5.25 triệu thấp hơn mức thực tế 7.93 triệu khoảng 33.8%.

Qua ví dụ này chúng ta thấy SHAP không nói mô hình đúng hay sai, nó chỉ nói mô hình đã dựa vào đâu. Với ngày này, mô hình nhận ra đúng tín hiệu cuối tháng và đẩy dự báo lên, nhưng nhóm cùng kỳ năm trước kéo mạnh hơn nên kết quả cuối cùng vẫn thấp. Muốn sửa thì phải sửa ở chỗ dữ liệu huấn luyện, chứ không phải ở chỗ giải thích.

IX. Kết luận

Tài liệu đã trình bày toàn bộ quy trình, từ xử lý tệp dữ liệu thô đến xây dựng mô hình có thể giải thích được. Từ 3,851 dòng chứa bốn loại lỗi, chúng ta thu được chuỗi gồm 3,833 ngày liên tục, xây dựng 53 đặc trưng thuộc bốn nhóm và huấn luyện năm mô hình cây. LightGBM đạt WAPE 0.2038 trên 184 ngày của tập kiểm tra, tốt hơn mô hình cơ sở 30.7%.

Thứ chi phối toàn bộ quy trình là mốc thông tin $d - H$. Nó quyết định đặc trưng nào được phép dựng, cách chia tập phải để đệm bao nhiêu ngày, và vì sao cột giá vốn, vốn có tương quan 0.976 với doanh thu, lại phải bị loại. Bốn trong sáu đặc trưng đứng đầu bảng SHAP đều xoay quanh mốc 364 ngày, cho thấy khi vùng gần bị chặn thì cùng kỳ năm trước là nguồn tín hiệu thay thế mạnh nhất.

Quy trình cũng cho thấy giá trị của việc kiểm tra tự động. Phép thử phá hoại ở Mục V.3 chạy trên 24 cột tại 8 mốc thời gian và cho kết quả bằng đúng 0, còn phép kiểm tính cộng tính ở Mục VIII.1 cho sai số 4.8×10^{-14} . Hai phép kiểm này chạy được mỗi lần sửa mã, nên chúng bắt lỗi sớm hơn nhiều so với việc đọc lại bằng mắt.

Tuy nhiên, chúng ta cũng cần nêu rõ ba hạn chế còn lại. Thứ nhất, mô hình dự báo thấp hơn thực tế 11.8% vì được huấn luyện bằng dữ liệu của ba năm có mức doanh thu thấp, rồi phải dự báo cho nửa năm đã hồi phục. Cách khắc phục là huấn luyện lại định kỳ, không phải nhân thêm một hệ số. Thứ hai, tập kiểm tra chỉ có 184 ngày nên chưa đủ để phân biệt dứt khoát bốn mô hình dẫn đầu, vốn chỉ cách nhau từ vài phần nghìn đến gần hai phần trăm WAPE; thứ hạng của chúng thay đổi theo cách gộp các lần cắt. Thứ ba, toàn bộ kết quả dựa trên một chuỗi duy nhất của một doanh nghiệp. Vì vậy, không nên áp dụng trực tiếp các con số cụ thể cho bài toán khác, mặc dù quy trình vẫn có thể được áp dụng.

Hướng phát triển tự nhiên là dự báo cho nhiều chuỗi cùng lúc thay vì một, thêm các nguồn dữ liệu ngoài như thời tiết hay giá của đối thủ, và thay dự báo điểm bằng dự báo khoảng để người dùng biết mức độ không chắc chắn. Cả ba hướng đều giữ nguyên ràng buộc về mốc thông tin.

Phụ lục

1. **Code - Data:** Các tệp dữ liệu và bài giải của Project 3.2 được lưu trong thư mục và có thể tải xuống [tại đây](#).

Lưu ý: Sau khi hết hạn nộp Project 3.2, giảng viên mới sao chép nội dung bài giải vào đường dẫn trên.

2. Rubric:

Mục	Kiến thức	Đánh giá
I	- Kiến thức về bài toán dự báo doanh thu theo chuỗi thời gian. - Khái niệm khoảng dự báo và mốc thông tin $d - H$. - Cấu trúc của hai tệp <code>sales.csv</code> và <code>promotions.csv</code>.	- Hiểu đúng mục tiêu dự báo doanh thu trước $H = 28$ ngày. - Xác định được thông tin nào có thể sử dụng tại thời điểm dự báo. - Phân biệt được biến mục tiêu, dữ liệu lịch sử và dữ liệu biết trước.
II–III	- Kiến thức về kiểu dữ liệu số và kiểu ngày tháng trong pandas. - Kiểm tra dữ liệu trùng, ngày thiếu và giá trị thiếu. - Chuẩn hóa định dạng ngày và dấu phân cách hàng nghìn.	- Phát hiện đúng các lỗi trong dữ liệu thô. - Chuyển đổi đúng các cột Revenue, COGS và Date. - Gộp các ngày trùng và tạo được chuỗi gồm 3,833 ngày liên tục.
IV	- Phân tích xu hướng, phân phối và các ngày có doanh thu cao. - Phân tích tính mùa vụ theo ngày trong tháng, tháng trong năm và thứ trong tuần. - Kiến thức về tự tương quan, Tết, chương trình khuyến mãi và COGS.	- Giải thích được các quy luật chính của chuỗi doanh thu. - Xác định được độ trễ 364 ngày là tín hiệu hợp lệ quan trọng. - Giải thích được vì sao không loại bỏ các ngày doanh thu cao và không dùng COGS cùng ngày.
V	- Kiến thức về đặc trưng trễ, thống kê trượt, đặc trưng lịch và đặc trưng Fourier. - Các đặc trưng liên quan đến Tết, ngày đặc biệt và chương trình khuyến mãi. - Khái niệm rõ ràng dữ liệu và phương pháp kiểm tra bằng phép thử rõ ràng.	- Xây dựng đúng 53 đặc trưng theo mốc thông tin $d - H$. - Không sử dụng doanh thu thuộc vùng bị chặn. - Kiểm tra được 24 đặc trưng lịch sử tại 8 mốc thời gian và xác nhận không có rõ ràng dữ liệu.
VI	- Chia tập huấn luyện, tập kiểm định và TEST theo thời gian. - Sử dụng vùng đệm giữa tập huấn luyện và tập kiểm định. - Kiến thức về WAPE, thiên lệch và R^2. - Các mô hình Decision Tree, Random Forest, Gradient Boosting, XGBoost và LightGBM.	- Thiết lập đúng các Fold và vùng đệm dài hơn khoảng dự báo. - Xây dựng mô hình cơ sở và so sánh bằng kiểm định chéo. - Chọn mô hình dựa trên tập kiểm định thay vì dựa trên TEST. - Đánh giá được đóng góp của từng nhóm đặc trưng.

VII	- Đánh giá mô hình trên TEST. - Phân tích WAPE, thiên lệch và R^2. - Khái niệm dịch chuyển phân phối giữa dữ liệu huấn luyện và giai đoạn dự báo.	- Đọc và so sánh đúng kết quả của năm mô hình. - Giải thích được thiên lệch âm của LightGBM. - Xác định được nguyên nhân dự báo thấp đến từ mức doanh thu năm 2022 phục hồi so với giai đoạn trước.
VIII	- Kiến thức về SHAP và TreeSHAP. - Mức quan trọng tổng thể của đặc trưng và cách giải thích cục bộ. - Tính cộng tính của giá trị SHAP.	- Xác định được các đặc trưng quan trọng nhất của LightGBM. - Giải thích được đóng góp của bốn nhóm đặc trưng. - Phân tích được một dự báo cụ thể bằng SHAP và kiểm tra tổng giá trị SHAP khớp với đầu ra của mô hình.
IX	- Tổng hợp kết quả, hạn chế và hướng phát triển của bài toán. - Huấn luyện lại mô hình khi dữ liệu thay đổi theo thời gian.	- Trình bày được kết quả chính của Project 3.2. - Nêu đúng các hạn chế của dữ liệu và TEST. - Đề xuất được hướng mở rộng phù hợp mà vẫn tuân thủ mốc thông tin $d - H$.